*free study notes*

# Phase 1
# Core Fundamentals

19 lessons · notes, diagrams and 152 interview questions

**1 · Watch**
Scan the QR code on a lesson to open its video.

**2 · Read**
Notes go deeper than the video: worked examples, trade-offs, mistakes.

**3 · Practise**
Answer the interview questions out loud before reading the answers.


| | |
|---|---|
| SD-A-001 · What Is System Design? HLD vs LLD Explained | Lesson 1 |
| SD-A-002 · Client-Server & Multi-Tier Architecture Explained (1, 2, 3 & N-Tier) | Lesson 2 |
| SD-A-003 · Functional Requirements in System Design (URL Shortener & Chat App Examples) | Lesson 3 |
| SD-A-004 · Non-Functional Requirements | Lesson 4 |
| SD-A-005 · System Design Trade-offs | Lesson 5 |
| SD-A-006 · Vertical Scaling Explained | Lesson 6 |
| SD-A-007 · Horizontal Scaling | Lesson 7 |
| SD-A-008 · Latency Explained | Lesson 8 |
| SD-A-009 · Throughput & Little's Law | Lesson 9 |
| SD-A-010 · Availability Explained | Lesson 10 |
| SD-A-011 · Reliability vs Availability vs Durability (Up ≠ Right) | Lesson 11 |
| SD-A-012 · Scalability Explained | Lesson 12 |
| SD-A-013 · CAP Theorem Explained (It's NOT "Pick Two") | Lesson 13 |
| SD-A-014 · PACELC Theorem | Lesson 14 |
| SD-A-015 · Consistency Models | Lesson 15 |
| SD-A-016 · Fault Tolerance | Lesson 16 |
| SD-A-017 · Single Point of Failure (SPOF) | Lesson 17 |
| SD-A-018 · Simple Beats Clever | Lesson 18 |
| SD-A-019 · Stateless vs Stateful | Lesson 19 |


Free to share. Videos: youtube.com/@tapankumarbarik9454 · Notes and practice: backendroadmap.com

# What Is System Design?

*HLD vs LLD, and what interviewers look for*

Watch the 2-min video

**In one line:** system design is turning requirements into an architecture: components, the data flows between them, and the trade-offs you choose to accept.

## 1 From requirements to an architecture

Every design starts from two kinds of requirements. **Functional** requirements say what the system must do: a user can shorten a URL, a driver can accept a ride. **Non-functional** requirements say how well it must do it: answer in under 200 ms, stay up 99.9% of the time, survive a data-centre outage, cost less than a set budget.

Design is the step from that list to a picture you can build: which **components** exist (services, databases, caches, queues), how **data flows** between them (who calls whom, synchronously or through a queue), and which **trade-offs** you accept to meet the numbers. Two engineers given the same requirements can draw different, equally valid designs; what makes one better is how well it fits the constraints, and how clearly its trade-offs are explained.

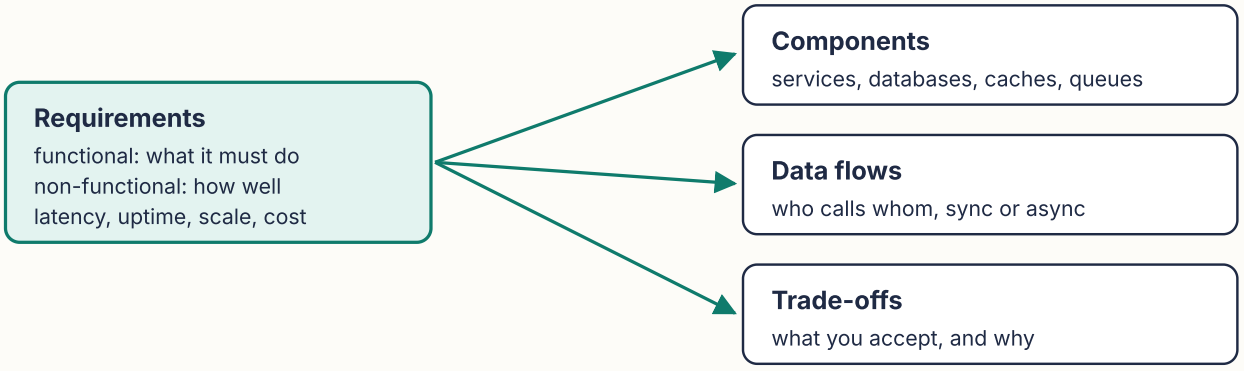



**Figure 1.** A design answers three questions about the requirements: what the parts are, how they talk, and what you give up.

## 2 Two levels: high-level and low-level design

**High-level design (HLD)** is the map of the system: services, data stores, networks, and how requests travel through them. It answers questions like “where is data stored?”, “what happens when this service fails?” and “how does it scale to 10× traffic?”.

**Low-level design (LLD)** zooms into one box of that map and designs its insides: classes, modules, interfaces, method signatures, and design patterns. It answers “how is this code organised so it is easy to test and change?”.

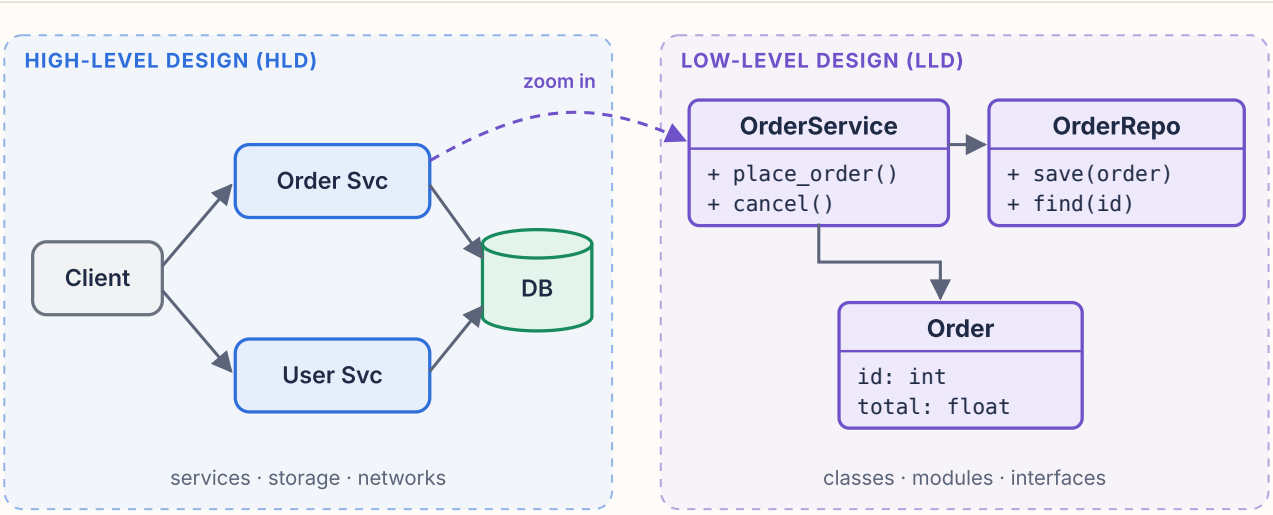



**Figure 2.** The same order service seen at two zoom levels: as one box in the system (left), and as classes inside that box (right).

| | High-level design (HLD) | Low-level design (LLD) |
|---|---|---|
| **Scope** | The whole system | One service or module |
| **Building blocks** | Services, databases, caches, queues, load balancers, CDNs | Classes, interfaces, functions, design patterns |
| **Typical questions** | Scale, availability, consistency, latency, cost | Responsibilities, extensibility, testability |
| **Output** | Architecture diagram, data flow, capacity estimates | Class diagram, API contracts, schema details |
| **Interview round** | “Design Twitter / a URL shortener” | “Design a parking lot / an LRU cache” |

## 3 Worked example: a FastAPI app grows up

Designs are rarely drawn complete on day one. They grow as load grows, one bottleneck at a time. Follow a small FastAPI app as its traffic rises.

1. **One server.** The app and its database share one machine. Cheap and simple, but the two compete for CPU, memory and disk, and one crash takes everything down.
2. **Separate the database.** The database moves to its own machine, sized for storage and memory. The app can now be restarted or redeployed without touching data.
3. **Scale the app tier.** A load balancer spreads requests over several identical copies of the app. One copy can fail without an outage, and you add copies as traffic grows. This works only if the app is *stateless* (no user session kept in a server's memory).
4. **Cache and CDN.** A cache such as Redis answers repeated reads in microseconds and protects the database. A CDN serves images, scripts and video from locations close to users.

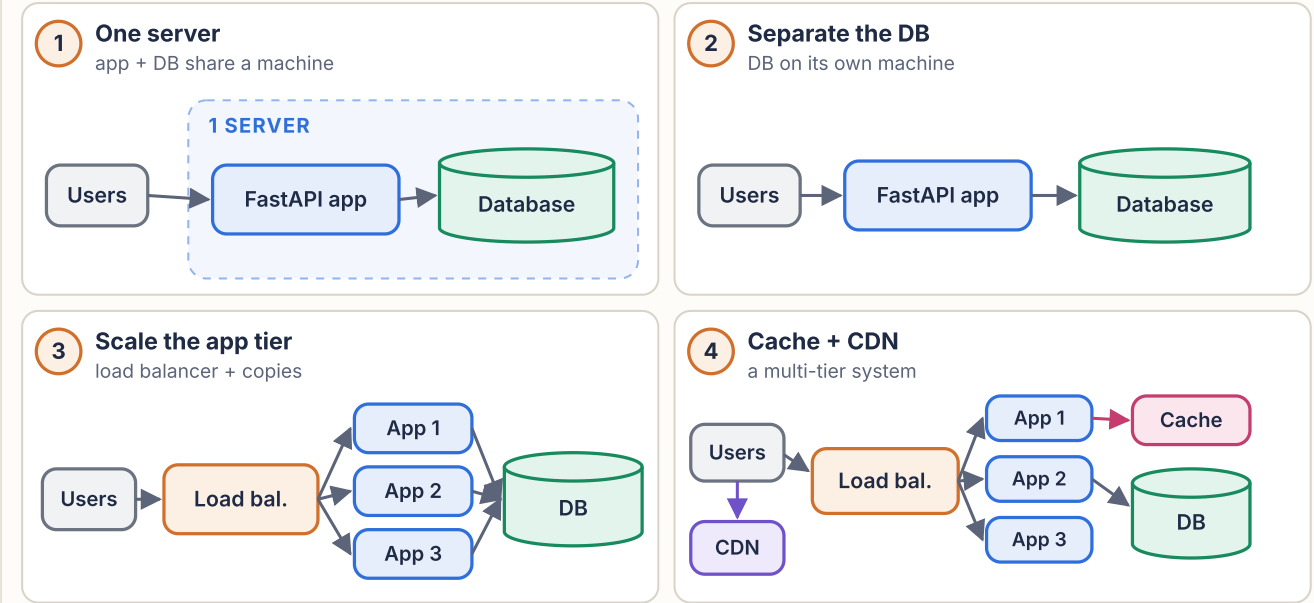



**Figure 3.** Four steps from one server to a multi-tier system. Each step fixes the bottleneck the previous one hit; in step 4, hot reads go to the cache and everything else to the database.

## 4 Every step is a trade-off

| Step | What you gain | What it costs |
|---|---|---|
| Separate database | Independent scaling, safer deploys | A network hop on every query, a second machine to run |
| Load balancer + copies | Higher availability, more throughput | More servers, stateless code required, more to monitor |
| Cache | Much lower read latency, less database load | Stale data risk, invalidation logic, another system to fail |
| CDN | Fast static content worldwide | Cost per GB, cache purging on every release |

**HOW TO TALK ABOUT TRADE-OFFS**

Name the option, the alternative, and the constraint that decides it: “I'll add a cache because reads outnumber writes 100 to 1 and the product page can tolerate data that is a few seconds old. If prices had to be exact, I would read prices from the database and cache only descriptions.”

## 5 What interviewers actually evaluate

A system design interview has no single right answer. The interviewer scores how you get to a design:

- **Structured thinking.** Clarify requirements and scale before drawing a box. Move from the big picture to the details.
- **Trade-offs.** For each choice, say why it beats the alternative here, and what would change your mind.
- **Communication.** Think out loud, check in with the interviewer, and adapt when they push on a part.

**A TYPICAL 45-MINUTE ROUND**

~5 min requirements and scope · ~5 min capacity estimates · ~10 min high-level design · ~15 min deep dives on 1–2 parts the interviewer picks · ~5 min bottlenecks, failures and wrap-up. Phase 8 of this course practises this exact structure.

**COMMON MISTAKES**

- Drawing boxes before asking what the system must do and at what scale.
- Naming technologies (Kafka, Cassandra) without saying why they fit.
- Designing for Google scale when the requirements say 1,000 users.
- Going silent for minutes; the interviewer can only score what you say.

## 6 Key terms

**Functional requirement**: what the system must do

**Non-functional requirement**: how well: latency, availability, cost

**HLD**: system-level architecture

**LLD**: component-level code design

**Load balancer**: spreads requests over servers

**Stateless service**: keeps no user state in memory, so any copy can serve any request

**Cache**: fast copy of hot data

**CDN**: servers near users for static content

## 7 Interview questions

**Q1. What does a system design interview evaluate?** BASIC

How you think, not one correct answer: whether you clarify requirements, structure the design from the big picture down, justify trade-offs against the constraints, and communicate while you work.

**Q2. How is high-level design different from low-level design?** BASIC

HLD covers the whole system: services, storage, networks and data flow, judged on scale, availability and cost. LLD covers the inside of one component: classes, interfaces and patterns, judged on clarity, extensibility and testability.

**Q3. Give functional and non-functional requirements for a URL shortener.** BASIC

Functional: create a short link, redirect to the long URL, optional custom alias and expiry. Non-functional: redirects in under ~100 ms at p99, very high availability for redirects (they are most of the traffic), hundreds of millions of links, and short codes that never collide.

**Q4. The interviewer says "Design Instagram." What do you do in the first five minutes?** SCENARIO

Clarify scope (photo upload, follow, feed; leave out DMs and stories unless asked), scale (daily users, uploads per day, read/write ratio), and non-functional goals (feed latency, availability, how fresh the feed must be). Confirm the list with the interviewer before drawing anything.

**Q5. Your single-server app is slowing down as traffic grows. What do you do first?** SCENARIO

Measure where the time goes. Usually: move the database to its own machine, then put a load balancer in front of several stateless app copies, then add a cache for hot reads. Each step targets the current bottleneck, not a guess.

**Q6. You add two more app servers behind a load balancer. What must change in the code?** SCENARIO

The app must become stateless: sessions move to Redis or signed tokens, uploaded files go to object storage instead of local disk, and in-memory caches can no longer be assumed shared. The load balancer also needs health checks so it stops sending traffic to a broken copy.

**Q7. Why is “add a cache” not a free improvement?** TRICKY

It trades freshness for speed: cached data can be stale, invalidation adds complexity, and the cache is one more component that can fail or fill up. It is worth it when reads dominate and slightly old data is acceptable.

**Q8. If there's no single right answer, how do you know a design is good?** TRICKY

It meets the stated requirements with numbers, has no unexplained single point of failure, every major choice comes with its reason and the alternative, and you can say what breaks first at 10× the load and how you'd fix it.

Next: SD-A-002 · Client-Server & Multi-Tier Architecture backendroadmap.com

# Client-Server & Multi-Tier Architecture

*why almost every app is split into layers*

Watch the 3-min video

**In one line:** a client asks, a server answers; splitting the server side into presentation, application and data tiers lets each part scale, change and be secured on its own.

## 1 Client and server

A **client** (a browser, a mobile app, another service) sends a **request**. A **server** does the work and sends back a **response**. The client never touches the server's internals; it only sees the API. That contract is what lets you change, scale or replace the server without shipping a new client.

## 2 One, two, three … N tiers

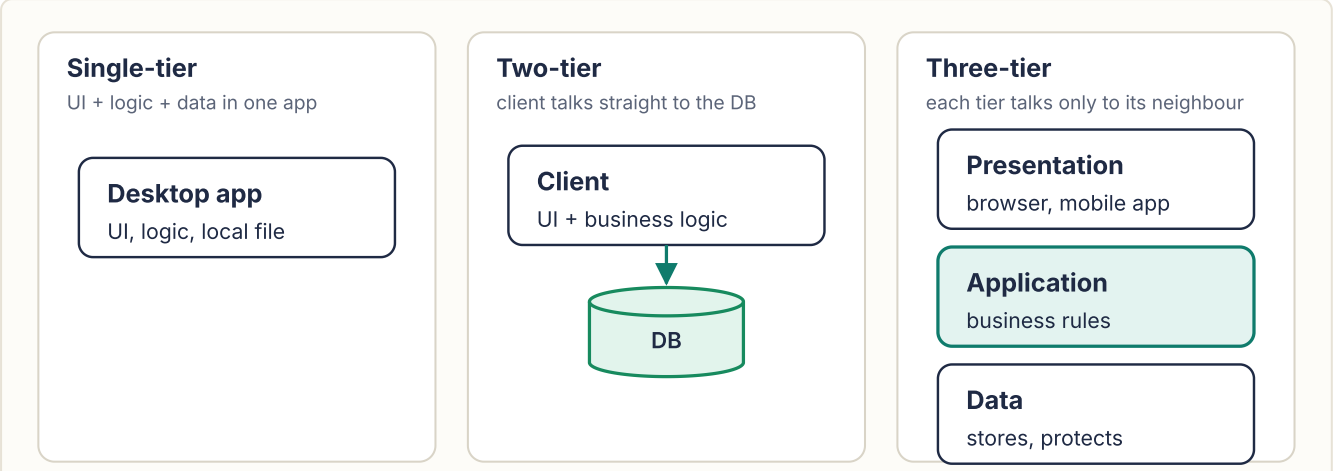
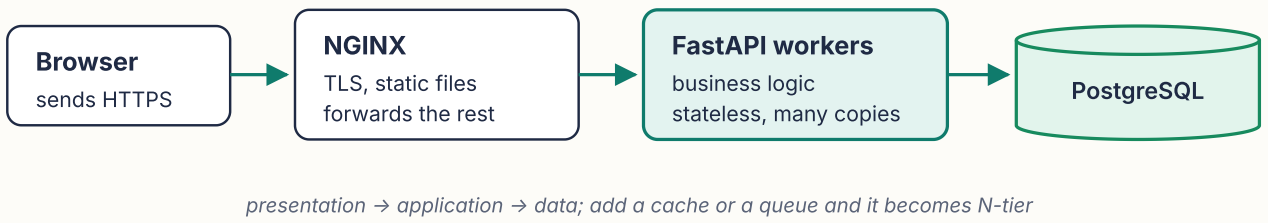



**Figure 1.** As tiers are added, business logic moves out of the client and the database stops being exposed to every user.

| Layout | Where the logic lives | Good for | Problem |
|---|---|---|---|
| **Single-tier** | One program on one machine | Desktop tools, scripts | Nothing is shared or scalable |
| **Two-tier** | In every client | Small internal tools | DB exposed to all clients; logic copied everywhere |
| **Three-tier** | The application tier | Most web and mobile apps | More moving parts to run |
| **N-tier** | App tier + caches, queues, workers | Large systems | Operational complexity, more hops |

## 3 A real request, tier by tier




*presentation → application → data; add a cache or a queue and it becomes N-tier*

**Figure 2.** A typical three-tier web stack. Only the application tier can reach the database.

1. **Browser → NGINX.** The reverse proxy terminates TLS, serves static files and forwards API calls.
2. **NGINX → FastAPI workers.** A pool of identical, stateless workers runs the business rules.
3. **Workers → PostgreSQL.** Reads and writes go to the data tier, which owns the state.

## 4 Why separate the tiers

- **Independent scaling.** The app tier is stateless, so you scale it *out* by adding workers. The data tier holds state, so it scales differently: a bigger machine, read replicas, or sharding.
- **Independent change.** Redesign the UI without touching the database; migrate the database without changing the UI.
- **Security.** Only the app tier can reach the database, so the data sits behind one controlled door instead of being open to every client.

**WORKED EXAMPLE: THE SLOWEST TIER WINS**

The database is overloaded and every page slows to 2 s. You double the app servers. Pages get *slower*: twice the workers open twice the database connections and add load to the bottleneck. The fix belongs in the data tier: a cache for hot reads, connection pooling, read replicas, or better indexes.

**TRADE-OFF: MORE TIERS VS FEWER HOPS**

Every tier adds a network hop (often 0.5–2 ms inside a data centre), another thing to deploy and another thing that can fail. Add a tier when it solves a real problem (scaling, security, caching), not by default.

**COMMON MISTAKES**

- Scaling the app tier when the database is the bottleneck.
- Keeping user sessions in an app server's memory, which breaks horizontal scaling.
- Letting clients talk to the database directly “just for this report”.

## 5 Key terms

**Client / server**: asks / answers

**Presentation tier**: what users see

**Data tier**: owns and protects state

**Tier**: a layer with one job, often its own machines

**Application tier**: business rules, stateless

**Reverse proxy**: front door: TLS, routing, static files

## 6 Interview questions

**Q1. Why separate the application tier from the data tier?** BASIC

So each can scale, change and be secured independently: stateless app servers scale out cheaply, the stateful database scales with replicas or sharding, and only the app tier can reach the data.

**Q2. What's the difference between two-tier and three-tier?** BASIC

In two-tier, clients talk to the database directly and carry the business logic. Three-tier puts an application server in between, so logic lives in one place and the database is no longer exposed.

**Q3. Why must the application tier be stateless to scale out?** BASIC

Any copy must be able to serve any request. If a session lives in one server's memory, a load balancer can't move that user elsewhere; keep state in a database, cache or signed token instead.

**Q4. Pages slow down when the database is overloaded. You add app servers. What happens?** SCENARIO

It gets worse: more app servers open more database connections and add load to the bottleneck. Measure, then fix the data tier with caching, pooling, read replicas or indexing.

**Q5. Where would you put a cache in a three-tier app?** SCENARIO

Between the application tier and the database for hot reads (Redis), and at the edge for static content (CDN). The app tier checks the cache first and falls back to the database.

**Q6. A mobile app needs data from three backend services. How do you avoid three round trips from the phone?** SCENARIO

Add a layer in front (an API gateway or a backend-for-frontend) that calls the services in parallel inside the data centre and returns one response.

**Q7. Every tier adds latency. When is a new tier not worth it?** TRICKY

When it doesn't solve a measured problem. Each hop adds network time, deployment work and a failure point; a small app with low traffic is often better as a simple three-tier stack.

**Q8. Is a microservices system still “three-tier”?** TRICKY

Each service is usually its own small multi-tier stack (API, logic, its own database). The system overall is N-tier: gateway, many services, caches, queues and data stores.

Next: SD-A-003 · Functional Requirements backendroadmap.com

# Functional Requirements

*what the system must do, and what to leave out*

Watch the 3-min video

**In one line:** functional requirements are the features, written as user actions; pick the 3–5 that make the product work, park the rest, and confirm the list before you design.

## 1 What they are

Functional requirements describe **what** the system does: its features. Not how fast or how big (that's non-functional, next lesson). Write them as **user actions**: “a user can create a short link”, “a user opening the link is redirected”. User actions are concrete, testable, and map straight to API endpoints.

## 2 From questions to a short list

1. **Ask first.** Who are the users? What must they be able to do? What does success look like?
2. **List the core use cases.** For a URL shortener: create a short link, redirect to the original URL.
3. **Add the optional extras the interviewer cares about:** a custom alias, an expiry date.
4. **Park the nice-to-haves.** Write them down so the interviewer sees you thought of them, then leave them out.
5. **Confirm.** “I'll focus on creating and redirecting, with custom aliases and expiry. Analytics are out of scope. Does that match what you want?”




**Figure 1.** A URL shortener's requirements, split into what you design now and what you park.

**THE RULE OF THUMB**

Keep **3–5 core use cases**. For each feature ask: without it, does the product still work? If yes, it's out of scope for now. A smaller scope lets you go deep on the hard parts in 45 minutes instead of drawing a shallow version of everything.

## 3 Requirements decide the architecture

Every requirement adds work to the design, so the list you agree on shapes everything after it.

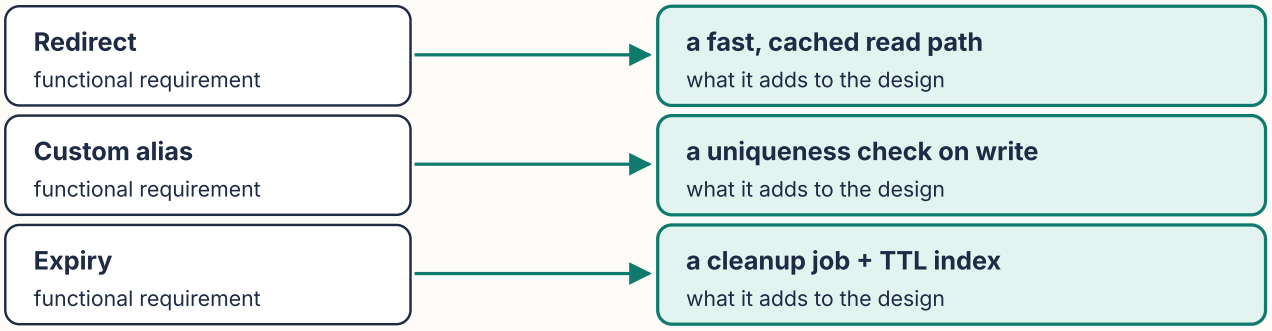



**Figure 2.** Each functional requirement becomes a concrete piece of the design.

## 4 Worked example: a chat app

CHAT APP, IN SCOPE

- Send and receive one-to-one messages
- Group chats
- Online status (presence)
- Message history
- Read receipts

**Confirm the edges:** "Are media uploads, voice or video calls in scope?" Usually they are not, and each would add a large subsystem (object storage, media processing, real-time media servers).

COMMON MISTAKES

- Drawing boxes before agreeing on features.
- Listing 15 features and designing none well.
- Mixing in non-functional goals ("must be fast") instead of user actions.
- Never confirming scope, then being asked about a feature you skipped.

## 5 Key terms

**Functional requirement**: a feature: what the system does

**Use case**: one user action and its result

**Scope**: what you will and won't design

**Nice-to-have**: noted, then parked

**Core use case**: without it, the product doesn't work

**Clarifying question**: turns a vague prompt into a list

## 6 Interview questions

**Q1. How do you decide which features are in scope?** BASIC

Ask clarifying questions, list the user actions, keep the 3–5 the product can't work without, park the rest out loud, and confirm the list with the interviewer.

**Q2. What's the difference between a functional and a non-functional requirement?** BASIC

Functional says what the system does ("send a message"). Non-functional says how well ("delivered in under 500 ms, 99.99% available"). The first drives the API and data model; the second drives the architecture's capacity and redundancy.

**Q3. List functional requirements for a chat app.** BASIC

One-to-one messages, group chats, online presence, message history, read receipts. Confirm whether media, voice and video are in scope; usually they are not.

**Q4. The prompt is just "Design YouTube." What do you ask?** SCENARIO

Who uploads and who watches? Must I cover upload, processing, playback, search, comments, recommendations? Which are core? Then agree on something like upload, transcode, stream and view counts, and park the rest.

**Q5. You agreed on "create and redirect." The interviewer later asks about analytics. What do you do?** SCENARIO

Show you parked it on purpose, then sketch how it plugs in without redesigning: emit a click event on each redirect to a queue, aggregate it asynchronously, and keep the redirect path fast.

**Q6. What does "custom alias" add to a URL shortener's design?** SCENARIO

A uniqueness check at write time (a unique index or conditional insert), a rule for reserved words, and a clear error when the alias is taken.

**Q7. Why not include every feature you can think of?** TRICKY

Time and depth. In 45 minutes a broad design stays shallow; interviewers score depth on the hard parts. A tight scope also makes the capacity estimates and trade-offs concrete.

**Q8. Is "the system must be secure" a functional requirement?** TRICKY

No, it's a quality attribute (non-functional). A functional version would be a specific action, like "only the owner can delete their link".

Next: SD-A-004 · Non-Functional Requirements backendroadmap.com

# Non-Functional Requirements

*how well the system must work, as numbers*

Watch the 3-min video

**In one line:** non-functional requirements are the quality targets (latency, availability, consistency, durability, scale, cost); written as numbers, they decide the architecture.

## 1 Same features, different systems

Two systems can have identical features and still need completely different architectures. A personal blog and a bank both “store and show data”; what separates them is **how well** they must do it. Those are the non-functional requirements (NFRs), also called quality attributes:

| Attribute | Question it answers | Example target |
|---|---|---|
| **Scale** | How much load? | 10,000 RPS peak, 500 M users |
| **Latency** | How fast for one request? | p99 < 200 ms |
| **Availability** | How often up? | 99.9% per month |
| **Consistency** | Must every read be the latest? | Strong for balances, eventual for likes |
| **Durability** | Can data ever be lost? | No acknowledged write lost |
| **Security, cost, compliance** | Who can access, at what price, under which rules? | Encrypted at rest, budget, GDPR |

## 2 Turn every adjective into a number

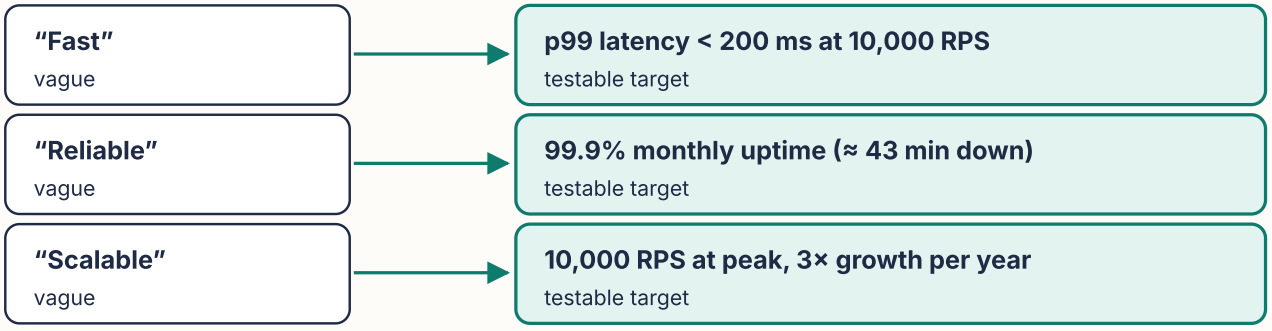
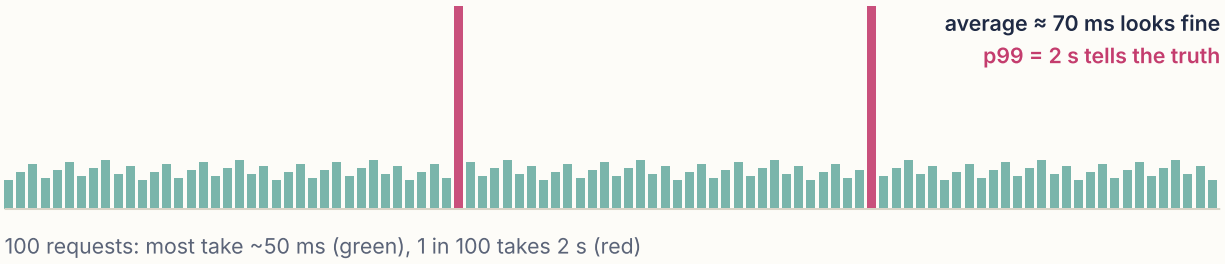



**Figure 1.** “Fast, reliable, scalable” mean nothing in a design review. A number can be tested and designed for.

## 3 Why p99, not the average




100 requests: most take ~50 ms (green), 1 in 100 takes 2 s (red)

**Figure 2.** The average hides the slow tail; p99 shows what 1 user in 100 waits.

A page often makes 20–50 backend calls, so a user hits the p99 latency of some call on almost every page load. That's why targets use percentiles (p50, p95, p99) measured at a stated load, not averages.

## 4 The nines, in downtime

| Availability | Downtime per year | Per month | Typically needs |
|---|---|---|---|
| **99%** | ~3.65 days | ~7.3 h | One server, restarts by hand |
| **99.9%** | ~8.8 h | ~43 min | Redundant servers, health checks |
| **99.99%** | ~53 min | ~4.3 min | Multiple zones, automatic failover |
| **99.999%** | ~5.3 min | ~26 s | Multiple regions, heavy automation |

Each extra nine costs much more to build and run, so ask what the business really needs.

## 5 Numbers change the architecture

- **High availability** → replicas across availability zones, automatic failover.
- **Low latency worldwide** → caches and a CDN near users, regional deployments.
- **Strong consistency** → a single source of truth or quorum writes; slower writes accepted.
- **High durability** → synchronous replication, backups, write-ahead logs.

**WORKED EXAMPLE: A PAYMENT SYSTEM**

The features are simple: charge a card, record the payment. The NFRs dominate: **strong consistency** (money is never double counted), **durability** (a confirmed payment is never lost), **idempotency** (a retried charge doesn't charge twice) and **auditability** (every change is traceable). Here correctness beats raw speed: 300 ms is fine, a lost payment is not.

**COMMON MISTAKES**

- Saying “fast and scalable” without numbers.
- Using averages for latency targets.
- Asking for five nines for an internal dashboard.
- Applying one consistency level to every feature.

## 6 Key terms

**NFR / quality attribute**: how well, not what

**p99 latency**: 99% of requests are faster

**SLO**: an internal target, e.g. 99.9%

**SLA**: a promise to customers, with penalties

**Durability**: acknowledged data is never lost

**Idempotency**: a retry has no extra effect

## 7 Interview questions

**Q1. Which non-functional requirements change the architecture most?** BASIC

Availability, latency, consistency, durability and scale, because each forces specific components: replicas and failover, caches and CDNs, quorums, synchronous replication, sharding.

**Q2. How do you quantify “fast”?** BASIC

As a percentile at a stated load: “p99 under 200 ms at 10,000 RPS”. Averages hide the slow tail that real users hit.

**Q3. How much downtime does 99.9% allow?** BASIC

About 43 minutes a month, or 8.8 hours a year. 99.99% allows about 4.3 minutes a month.

**Q4. What NFRs would you set for a URL shortener's redirect path?** SCENARIO

Very high availability (redirects are almost all traffic and links live in emails and printed media), low latency (p99 under ~100 ms), and high read scale; eventual consistency for a newly created link is usually fine.

**Q5. Product says “it must never be down.” How do you respond?** SCENARIO

Turn it into a number with a cost: 99.99% means multi-zone redundancy and automatic failover; 99.999% means multi-region and much more spend. Ask what an hour of downtime costs the business, then pick.

**Q6. List the NFRs for a payment service.** SCENARIO

Strong consistency, durability of confirmed payments, idempotent APIs, auditability, high availability for accepting payments, PCI-DSS compliance; moderate latency is acceptable.

**Q7. Why can a service with p99 = 100 ms still feel slow?** TRICKY

A page makes many calls, often in sequence. With 30 calls, the chance at least one hits its p99 is about 26%, and sequential calls add up. Use latency budgets per call and parallelise.

**Q8. Can one system have different consistency targets?** TRICKY

Yes, and it usually should. A shop needs exact stock and payments (strong) but can show reviews and recommendations a few seconds late (eventual).

Next: SD-A-005 · Trade-offs backendroadmap.com

# Trade-offs

*how to justify any design choice*

Watch the 3-min video

**In one line:** every design choice improves one quality and costs another; a senior answer names the choice, the reason from the requirements, what you accept, and how you limit it.

## 1 There is no free lunch

The same tensions come up in almost every design. Knowing them by name lets you spot them early and talk about them clearly.

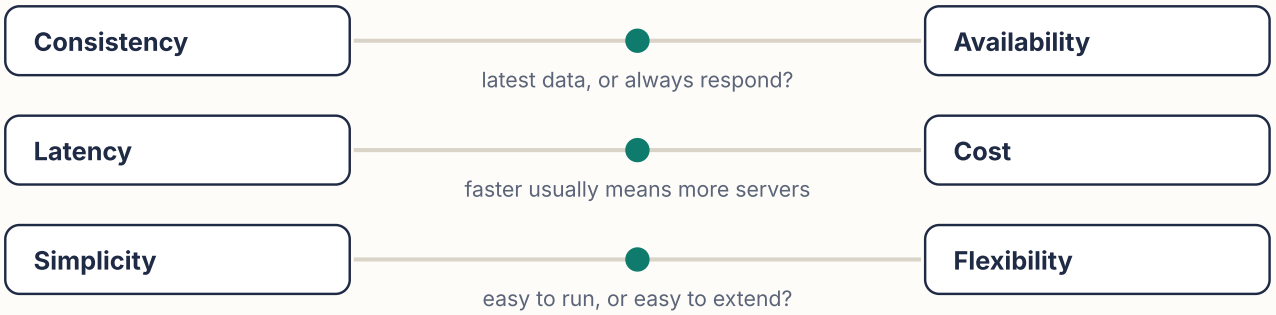



**Figure 1.** Three trade-offs you will meet in nearly every interview. The requirements decide where the dot sits.

## 2 A worked example

**PRODUCT PAGE AND A CACHE**

A product page reads from the database on every request, and the database is struggling. You add a cache with a 60-second expiry. Database load drops by ~80%. The price: a user might see a price up to a minute old.

Now say it out loud with a four-part template:

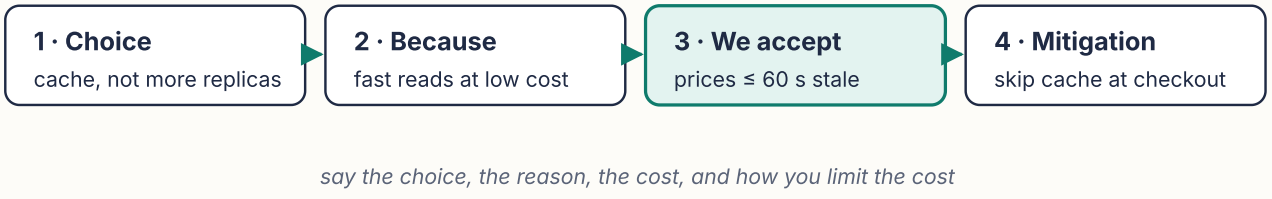



**Figure 2.** The four-part trade-off statement. Part 3 is what interviewers listen for: you know what you gave up.

*"We chose a cache over more read replicas, because the requirement is fast reads at low cost. We accept prices up to 60 seconds stale, and we bypass the cache at checkout, where the price must be exact."*

## 3 From “it depends” to a decision

“It depends” is a fine start but not an answer. Say **what** it depends on, then commit:

1. List the quality attributes that matter (latency, consistency, cost ...).
2. Rank them using the requirements.
3. Score each option against them.
4. Pick, and state what would change your mind.

| Option | Read latency | Freshness | Cost | Verdict |
|---|---|---|---|---|
| **Cache (60 s TTL)** | ~1 ms | ≤ 60 s stale | Low | ✓ for product pages |
| **More read replicas** | ~5–10 ms | Replication lag, usually < 1 s | Higher | If freshness matters more |
| **Bigger primary DB** | ~5–10 ms | Exact | Highest, has a ceiling | Short-term relief |

This is the idea behind ATAM-style reasoning (Architecture Trade-off Analysis Method): make the attributes explicit and score options against them.

## 4 Justifying eventual consistency

Ask what happens if a user sees slightly old data. For a like counter or a feed, a few seconds of delay is harmless, and you gain availability and lower latency. For a bank balance it is not. Add guardrails where needed, like **read-your-writes**: a user always sees their own changes immediately, even if others see them a moment later.

TALKING ABOUT A REAL PROJECT (STAR-STYLE)

**Context** → **options** you considered → **choice and its cost** → **result with a number**. “We cached repeated LLM answers, cut model cost by 40%, and skipped the cache for time-sensitive questions.”

COMMON MISTAKES

- Presenting a choice as having no downside.
- Stopping at “it depends”.
- Naming a technology instead of the property you need from it.
- One consistency level for every feature.

## 5 Key terms

**Trade-off**: gain one quality, pay in another

**Eventual consistency**: replicas converge, reads may lag

**ATAM**: scoring options against ranked attributes

**Quality attribute**: latency, availability, cost ...

**Read-your-writes**: you always see your own changes

**Mitigation**: how you limit the cost you accepted

## 6 Interview questions

**Q1. How do you justify a design choice?** BASIC

Name the choice, the requirement that drives it, what you give up, and how you limit that cost. Then say what would make you choose differently.

**Q2. When is eventual consistency acceptable?** BASIC

When stale data is harmless for that feature (likes, feeds, view counts), and you gain availability or latency. Add read-your-writes so users see their own changes.

**Q3. What's wrong with answering “it depends”?** BASIC

Nothing as a start, but it isn't an answer. Say what it depends on (the ranked requirements), score the options, and commit.

**Q4. Describe a real trade-off from your work.** SCENARIO

Use context, options, choice and cost, result with a number. For example: cached repeated LLM answers, cut cost by 40%, accepted stale answers for 1 hour, bypassed the cache for time-sensitive queries.

**Q5. Cache or read replicas for a read-heavy product catalogue?** SCENARIO

A cache with a short TTL if slightly stale data is fine: cheaper and ~1 ms reads. Read replicas if data must be fresher or queries are too varied to cache. Often both: replicas behind a cache.

**Q6. Your team wants microservices for a 3-person startup. How do you respond?** SCENARIO

Weigh flexibility against operational cost. With 3 engineers, a modular monolith is usually faster to build and run; split services later where scaling or team boundaries demand it.

**Q7. A like counter shows 1,203 to one user and 1,207 to another. Is that a bug?** TRICKY

Not if the requirement allows eventual consistency for likes. It becomes a bug only if the product needs exact counts (e.g. billing by likes), then you need strongly consistent counting.

**Q8. How do you know which trade-off the interviewer wants you to make?** TRICKY

You don't guess: you derive it from the requirements you agreed on, state it, and offer the alternative. Interviewers score the reasoning, not a specific pick.

Next: SD-A-006 · Vertical Scaling backendroadmap.com

# Vertical Scaling

*make one machine bigger, and know when to stop*

Watch the 3-min video

**In one line:** vertical scaling (scaling up) gives one machine more CPU, memory or faster disks: no code changes and quick relief, but it hits a ceiling, gets expensive and stays a single point of failure.

## 1 What it is

Scaling up means making one machine more powerful: more CPU cores, more RAM, faster storage (for example moving to NVMe SSDs), a faster network card. Your code doesn't change; the same app simply runs on stronger hardware. In the cloud it's often one setting: change the instance type and restart.

## 2 A worked example




**Figure 1.** A database at its limit, fixed by resizing the machine.

The PostgreSQL VM has 8 vCPUs and sits at 90% CPU at peak. Resizing to 32 vCPUs and more memory brings CPU to ~30%, and more of the working set now fits in memory, so queries hit the disk less. Four times the cores rarely means four times the speed (locks, single-threaded parts and I/O limit the gain), but it buys a lot of headroom fast.

## 3 The four limits

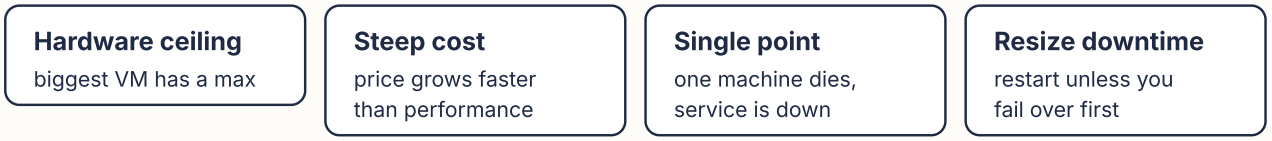



**Figure 2.** Why scaling up is a first step, not a final answer.

- **Hardware ceiling.** Even the largest instance has a maximum number of cores and memory. Past it, you can't go higher.
- **Steep cost curve.** The biggest and high-memory machines carry a premium, while software rarely uses every extra core well. Each extra unit of performance costs more.
- **Single point of failure.** Everything still lives on one machine. If it fails, the service is down.

- **Downtime to resize.** Changing size usually means a restart, unless you first fail over to a replica and resize the old primary.

## 4 When it's the right move

| Situation | Why scaling up fits |
|---|---|
| You need relief today | One setting, no code or data changes |
| Stateful, hard-to-split workloads | A relational database with joins and transactions works best on one node |
| You're still small | One big machine is simpler than a cluster to run |
| Buying time | Gives you months to plan replicas, caching or sharding properly |

**SCALE UP VS SCALE OUT**

Scale up first for stateful parts and early-stage systems; scale out for stateless tiers and when you need fault tolerance or growth beyond one machine. Most real systems do both: a big primary database plus many small app servers.

**COMMON MISTAKES**

- Dismissing vertical scaling as “not scalable”: it's often the cheapest first fix.
- Scaling up forever and discovering the ceiling during a traffic spike.
- Forgetting it doesn't improve availability.
- Resizing the primary without a replica to fail over to.

## 5 Key terms

**Vertical scaling**: a bigger machine (scale up)

**vCPU**: a virtual CPU core in the cloud

**Failover**: switching to a standby

**Horizontal scaling**: more machines (scale out)

**Working set**: data accessed often; best kept in RAM

**Headroom**: spare capacity for spikes

## 6 Interview questions

**Q1. When would you scale vertically instead of horizontally?** BASIC

Early on, for stateful systems like a relational database, and when you need a quick fix without code changes. It buys time to plan horizontal scaling properly.

**Q2. What are the limits of vertical scaling?** BASIC

A hardware ceiling, a steep cost curve, a single point of failure, and downtime during resize unless you fail over first.

**Q3. Does vertical scaling improve availability?** BASIC

No. It's still one machine, so one failure takes the service down. Availability needs redundancy: a replica and failover.

**Q4. Your database is at 90% CPU at peak. What do you do this week?** SCENARIO

Check for slow queries and missing indexes first (often the cheapest fix), then resize the instance via a replica failover. In parallel, plan read replicas or caching for growth.

**Q5. How do you resize a production database with minimal downtime?** SCENARIO

Create or use a replica, resize it, promote it to primary (a few seconds of failover), then resize or replace the old primary as the new replica.

**Q6. Doubling the cores only gave 30% more throughput. Why?** SCENARIO

Something else is the bottleneck: lock contention, a single-threaded part, disk I/O or the network. Amdahl's Law: the serial part limits the gain from more parallel hardware.

**Q7. Is “just get a bigger machine” ever the best senior answer?** TRICKY

Yes, when it's the fastest, cheapest way to meet the requirement and the growth forecast fits under the ceiling. Say so, and name the trigger for moving to horizontal scaling.

**Q8. Why are high-memory instances so expensive per GB?** TRICKY

They're specialised hardware made in smaller volumes, and cloud pricing at the top end includes a premium. Performance per dollar falls as size grows.

Next: SD-A-007 · Horizontal Scaling backendroadmap.com

# Horizontal Scaling

*add more servers instead of a bigger one*

Watch the 3-min video

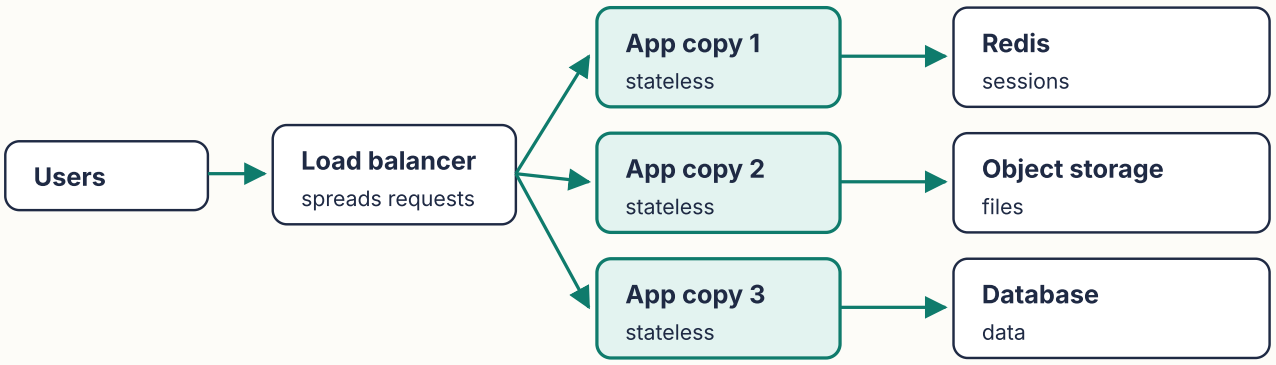
**In one line:** horizontal scaling (scaling out) runs many copies behind a load balancer; it only works when the service is stateless, and it's much harder for databases because they hold state.

## 1 Scale out, not up

One machine always hits a ceiling. Scaling out runs many copies of a service side by side, with a load balancer in front spreading requests across them. Need more capacity? Add a copy. Traffic drops? Remove one. There's no hard ceiling, and losing one copy doesn't take the service down.

## 2 The rule: be stateless

Imagine a server keeps a user's session in its own memory. The next request lands on a different server, the session isn't there, and the user is logged out. The fix is to move state out of the servers: sessions to a shared store like Redis (or a signed token), files to object storage, data to the database.




*state lives outside the copies, so any copy can serve any request*

**Figure 1.** A horizontally scaled service: identical, stateless copies behind a load balancer, with state in shared stores.

## 3 A real example: autoscaling on Kubernetes

**ELASTIC FASTAPI SERVICE**

Twenty FastAPI pods run behind a Kubernetes Service. A Horizontal Pod Autoscaler (HPA) watches average CPU, targeting, say, 60%. When traffic spikes and CPU rises, it adds pods; when traffic falls, it removes them (with a cooldown so it doesn't flap). This ability to grow and shrink with demand is **elasticity**.

## 4 What changes when you scale out

| | Effect |
|---|---|
| **Availability** ↑ | One copy dies, the others keep serving (if health checks remove it quickly) |
| **Cost** flexible | Pay for what you run; scale down at night |
| **Complexity** ↑ | Load balancing, health checks, service discovery, config and deploys across many machines |
| **Debugging** harder | A request may touch any copy; you need central logs and tracing |

## 5 Why databases are harder to scale out

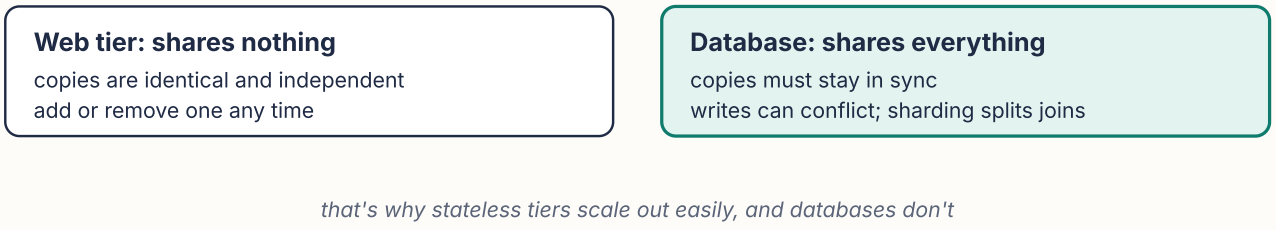



**Figure 2.** Stateless web servers share nothing. Databases share everything.

Read load can scale with **read replicas** (copies that follow the primary). Write load needs **sharding**: splitting data across machines by a key. Then queries that join across shards, and transactions that touch several shards, become slow or complicated. That's why teams scale the database up first, and shard only when they must.

COMMON MISTAKES

- Scaling out a service that keeps sessions in memory.
- Relying on sticky sessions as the fix (uneven load, lost sessions on failure).
- Autoscaling on CPU when the real bottleneck is the database.
- Forgetting the database connection limit when adding 50 app copies.

## 6 Key terms

**Horizontal scaling**: more machines (scale out)

**Elasticity**: grow and shrink with demand

**Read replica**: a copy for reads

**Stateless**: no user state kept between requests

**HPA**: Kubernetes autoscaler for pods

**Sharding**: splitting data across machines by a key

## 7 Interview questions

**Q1. What must be true before a service can scale horizontally?** BASIC

It must be stateless: sessions, files and data live in shared stores, so any instance can serve any request and instances can be added or removed freely.

**Q2. Why is scaling a database horizontally harder?** BASIC

State must be replicated (kept in sync) or partitioned (sharded) while staying consistent; cross-shard joins and transactions become hard. Web servers share nothing.

**Q3. What is elasticity?** BASIC

Adding capacity when load rises and removing it when load falls, automatically, so you pay for what you use.

**Q4. Users get logged out randomly after you add servers. Why, and how do you fix it?** SCENARIO

Sessions are stored in each server's memory, so a request on another server finds none. Move sessions to Redis or use signed tokens; sticky sessions only hide the problem.

**Q5. You scale the API from 10 to 50 pods and the database falls over. What happened?** SCENARIO

Each pod opens its own connection pool, so connections jumped 5× past the database's limit. Use a connection pooler (e.g. PgBouncer), smaller per-pod pools, and cache reads.

**Q6. Which metric would you autoscale a queue worker on?** SCENARIO

Queue depth or message age, not CPU: they measure the backlog directly. Workers waiting on I/O can have low CPU while the queue grows.

**Q7. Are sticky sessions a good way to scale a stateful app?** TRICKY

Only as a short-term patch: load becomes uneven, sessions are lost when a server dies, and deploys get harder. Externalise the state instead.

**Q8. Can you scale writes with read replicas?** TRICKY

No. Replicas copy the primary's writes, so every write still goes through the primary. Write scaling needs sharding or a database built to partition writes.

Next: SD-A-008 · Latency backendroadmap.com

# Latency

*p99, latency budgets and the numbers to know*

Watch the 3-min video

**In one line:** latency is the time for one request, made of network, queueing, processing and I/O; measure it as percentiles, know the key numbers, and split the target into a budget per component.

## 1 What latency is made of

Latency is the time from sending a request to receiving the full response. It's never one thing; it's the sum of four parts, and each part has different fixes.




**latency = network + queueing + processing + I/O**

**Figure 1.** The four parts of latency. Queueing is the sneaky one: it grows fast near full utilization, even when your code doesn't get slower.

## 2 Measure percentiles, not averages

**p50** is the typical request; **p95** and **p99** are the slow tail. If one page calls 10 services, the chance that at least one call is at or beyond its p99 is $1 - 0.99^{10} \approx$ **10%**. With 50 calls it's ~40%. The tail is what users actually feel.

## 3 Numbers every engineer should know

| Operation | Typical time | Relative to memory |
|---|---|---|
| Read from main memory | ~100 ns | 1× |
| Random read from an SSD | ~100 µs | ~1,000× |
| Round trip inside one data centre | ~0.5 ms | ~5,000× |
| Read 1 MB sequentially from an SSD | ~1 ms | ~10,000× |
| Round trip across regions (e.g. India ↔ Europe) | 100–150 ms | ~1,000,000× |

Rough orders of magnitude are enough: they tell you instantly that a cross-region call inside a loop is a bad idea, and that caching in memory beats any disk.

## 4 Latency budgets

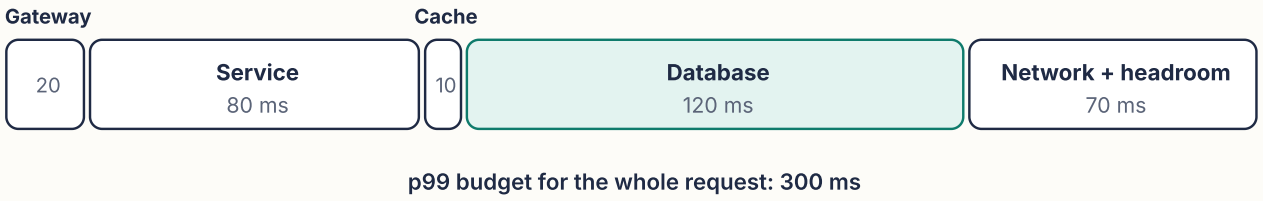



**Figure 2.** A 300 ms p99 target split into a budget per component. Every team knows its number.

If a part goes over budget, you know where to look, and each fix maps to one part of latency:

| Too much ... | Fix |
|---|---|
| **Network** | Move closer to users (CDN, regions), cut round trips, batch calls |
| **Queueing** | Add capacity, lower utilization, shed low-priority load |
| **Processing** | Profile hot code, do less work per request, move work to async jobs |
| **I/O / database** | Add an index, cache hot reads, avoid N+1 queries |

COMMON MISTAKES

- Reporting average latency.
- Calling services one after another when they could run in parallel.
- Ignoring cross-region round trips in the design.
- Measuring latency without stating the load it was measured at.

## 5 Key terms

**Latency**: time for one request

**Tail latency**: the slowest few percent

**Latency budget**: max time per component

**p99**: 99% of requests are faster

**Queueing delay**: time waiting for a free worker

**RTT**: round-trip time

## 6 Interview questions

**Q1. Why use p99 instead of the average?** BASIC

Averages hide the slow tail, and pages make many calls, so most users hit some call's p99 regularly. Percentiles show what users actually feel.

**Q2. How does a cross-region round trip compare with a memory read?** BASIC

Roughly 100 ms versus 100 ns: about a million times slower.

**Q3. What are the parts of latency?** BASIC

Network time, queueing (waiting for a worker), processing (your code) and I/O (disks, databases). Each has different fixes.

**Q4. Your API's p99 jumped from 120 ms to 900 ms but code didn't change. Where do you look?** SCENARIO

Queueing and dependencies: higher utilization, a saturated pool or a slow database. Check traffic, CPU, pool wait time and database latency, using traces to see which span grew.

**Q5. Split a 300 ms p99 budget for a checkout request.** SCENARIO

Gateway ~20 ms, service logic ~80, cache ~10, database ~120, and ~70 for network and headroom. Then each team owns its number.

**Q6. A page calls 5 services one after another, each 60 ms. How do you speed it up?** SCENARIO

Call independent services in parallel (≈ 60 ms instead of 300), cache results that change rarely, or have one aggregator call them inside the data centre.

**Q7. With 10 calls per page, why do ~10% of pages hit a p99?** TRICKY

Each call has a 1% chance of being in its slowest 1%. The chance at least one of 10 is: $1 - 0.99^{10} \approx 9.6\%$.

**Q8. Can adding a cache make p99 worse?** TRICKY

Yes, if misses add a network hop before the database, or the cache is overloaded or cold after a restart. Measure hit rate and miss latency, and warm caches after deploys.

Next: SD-A-009 · Throughput & Little's Law backendroadmap.com

# Throughput & Little's Law

*how much work, how fast, and how many at once*

Watch the 3-min video

**In one line:** throughput is work done per second; Little's Law ties it to latency (in flight = throughput × latency), and queues make latency explode as a system nears 100% busy.

## 1 Latency and throughput are different questions

Picture a highway. **Latency** is how long one car takes to make the trip. **Throughput** is how many cars get through per hour. A road can be fast for each car but carry few of them, if it has only one lane.

Throughput is work completed per unit of time, in the unit each part speaks: **RPS** (requests per second) for APIs and load balancers, **QPS** (queries per second) for databases and caches, **MB/s** or **Gbps** for networks and disks, **messages/s** for queues and streams.

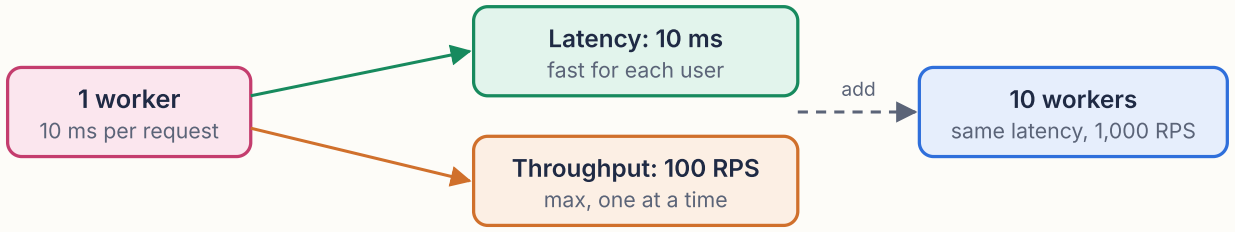



**Figure 1.** One worker at 10 ms per request can never serve more than 100 requests per second; more workers raise throughput, not latency.

## 2 Little's Law: the link between them

The bridge between throughput and latency is **concurrency**: how many requests are in flight at the same moment. Little's Law says that, for any stable system:

$$L = \lambda \times W \quad \cdot \quad \text{in flight} = \text{throughput} \times \text{latency}$$

It holds for any system that is not piling up work without limit: a web server, a database connection pool, a checkout line, a Kafka consumer group. It is the fastest way to size anything that holds requests at once: worker threads, connection pools, queue capacity.

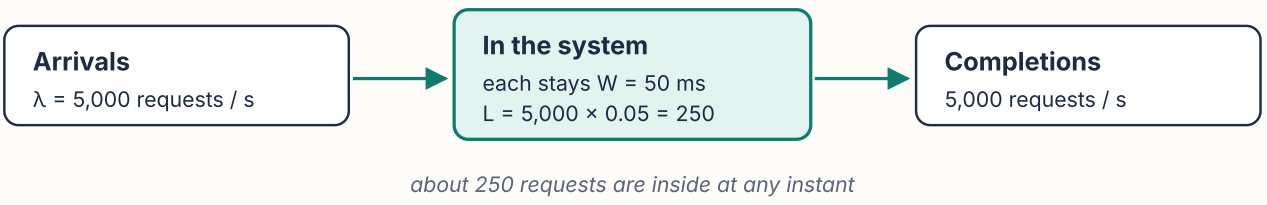



**Figure 2.** 5,000 requests arrive per second and each stays 50 ms, so about 250 are inside the system at any instant.

**WORKED EXAMPLE**

An API handles **5,000 RPS** and each request takes **50 ms** (0.05 s). In flight: 5,000 × 0.05 = **250**. Your workers, threads and database pool together must hold at least 250 concurrent requests, plus headroom.

Now the database slows down and latency doubles to 100 ms. At the same traffic, in-flight requests double to **500**. A pool sized for 250 runs dry, new requests wait for a connection, latency rises further, and a small slowdown becomes an outage.

Same rule, a database pool: 2,000 queries/s at 20 ms each needs about 2,000 × 0.02 = **40** connections.

## 3 Saturation: throughput hits a ceiling

As load rises, throughput first rises with it. Every system has a capacity, set by its slowest resource (CPU, disk, a lock, a pool). Past that point throughput stays flat: extra requests cannot be served faster, so they wait in a queue. And waiting is where latency goes.

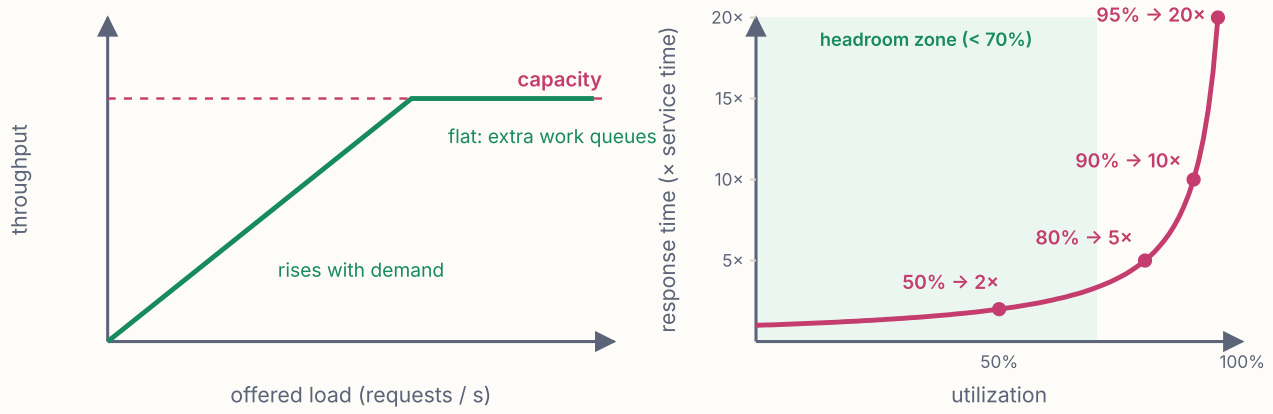



**Figure 3.** Left: throughput flattens at capacity. Right: response time in a simple queue grows like 1 / (1 − utilization), so it explodes near 100% (at 99% it reaches 100×, off this chart).

That is why teams run servers at 50–70% at peak, autoscale before 80%, and alert well before 100%. The last 20% of capacity costs most of your latency.

## 4 How to raise throughput

- **Find the bottleneck first.** Measure which resource saturates (CPU, I/O, a lock, a pool). Scaling anything else just moves the queue.
- **Scale out.** Add parallel workers or servers behind a load balancer (needs stateless services).
- **Batch.** Group small operations: one insert of 500 rows beats 500 inserts; one network call for 50 keys beats 50 calls.
- **Async, non-blocking I/O.** Workers serve other requests while waiting on the network or disk, instead of sitting idle.
- **Load test.** Find the highest throughput you can sustain *while still meeting your p99 latency target*. That number, not the peak, is your capacity.
- **Mind the trade-off.** Batching and big queues raise throughput but add waiting time: protect p99 for user-facing APIs, maximise throughput for batch jobs.

**COMMON MISTAKES**

- Treating “fast” as “handles a lot”: they are different numbers.
- Sizing pools from averages; size for peak traffic and slow-day latency.
- Planning to run at 95% utilization to save money; latency will be unusable.

## 5 Key terms

**Throughput**: work completed per second (RPS, QPS, MB/s)

**Latency**: time for one request

**Concurrency**: requests in flight at once

**Little's Law**: L = λ × W

**Utilization**: share of time a resource is busy

**Saturation**: load beyond capacity; work queues

## 6 Interview questions

**Q1. What is Little's Law, and how do you use it?** BASIC

In flight = throughput × latency (L = λW). Use it to size anything that holds requests at once. 5,000 RPS at 50 ms means about 250 requests in flight, so workers, threads and pools need room for 250 plus headroom.

**Q2. Can a system have low latency but low throughput?** BASIC

Yes. One worker that answers in 10 ms is fast for each user but can never exceed 100 requests per second. Latency is time per request; throughput is requests per second; concurrency links them.

**Q3. Why does latency rise sharply near 100% utilization?** BASIC

Requests start queueing. In a simple queue model, response time grows like 1 / (1 − utilization): about 2× at 50%, 10× at 90% and 100× at 99%.

**Q4. Your API does 2,000 RPS at 40 ms. After a deploy, latency jumps to 200 ms. What happens?** SCENARIO

In-flight requests rise from 2,000 × 0.04 = 80 to 2,000 × 0.2 = 400. Pools sized for ~100 run out, requests wait, latency climbs further and errors follow. Roll back, then find the slow dependency.

**Q5. Size a database connection pool for 3,000 queries/s at 15 ms each.** SCENARIO

3,000 × 0.015 = 45 connections on average. Add headroom for bursts (about 1.5–2×, so 70–90) and check the total across all app instances against the database's connection limit: 10 instances × 9 connections each, not 10 × 90.

**Q6. How would you increase a service's throughput?** SCENARIO

Find the bottleneck first, then fix that resource: scale out, batch, async I/O, cache. Confirm with a load test at the p99 target.

**Q7. Why plan servers to run at 60–70% at peak instead of 95%?** TRICKY

Queueing: at 95%, latency is about 20× the service time. Failures: with three servers at 70%, losing one pushes the other two to about 105%; at 95%, one failure means overload.

**Q8. Average latency is 50 ms but p99 is 2 s. Which do you use with Little's Law?** TRICKY

Little's Law uses the mean, which gives average concurrency. But the slow tail holds connections 40× longer, so add headroom and set timeouts so slow requests can't drain the pool.

Next: SD-A-010 · Availability Explained backendroadmap.com

# Availability

*the nines, MTBF vs MTTR, series vs parallel*

Watch the 3-min video

**In one line:** availability is the share of time a system serves requests successfully; it depends on how often things fail and how fast you recover, and it multiplies down in series and up with redundancy.

## 1 The nines

| Availability | Name | Downtime per year | Per month |
|---|---|---|---|
| 99% | two nines | ~3.65 days | ~7.3 h |
| 99.9% | three nines | ~8.76 h | ~43.8 min |
| 99.99% | four nines | ~52.6 min | ~4.4 min |
| 99.999% | five nines | ~5.3 min | ~26 s |

## 2 MTBF and MTTR

Two numbers drive availability: **MTBF** (mean time between failures: how often things break) and **MTTR** (mean time to recovery: how fast you fix them).

```
availability = MTBF / (MTBF + MTTR)
```

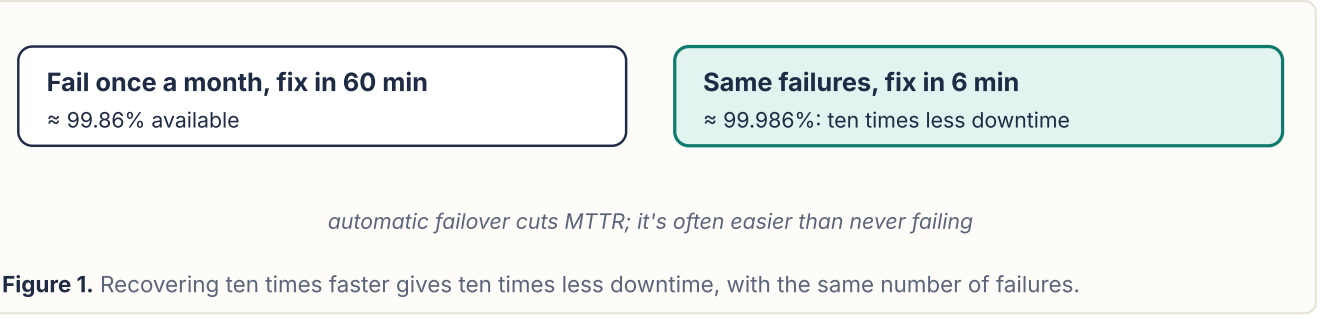



**Figure 1.** Recovering ten times faster gives ten times less downtime, with the same number of failures.

## 3 Series and parallel

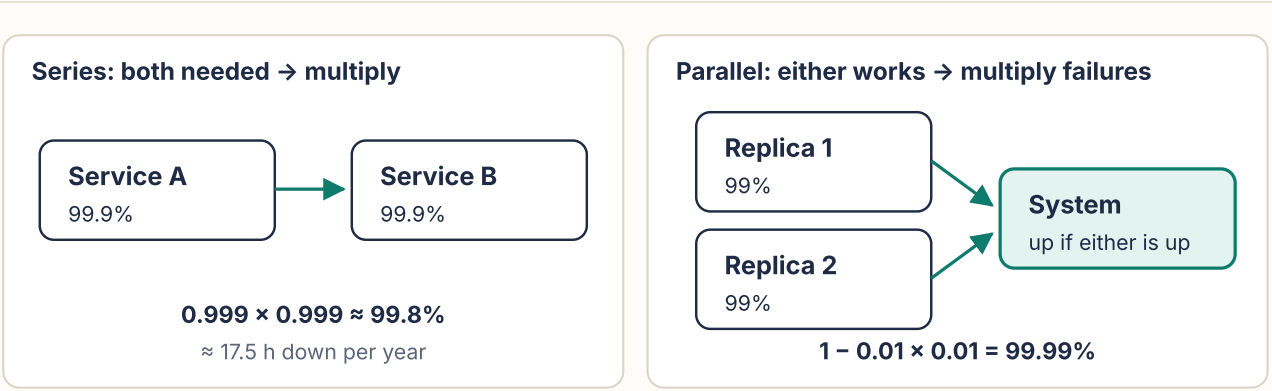



**Figure 2.** Dependencies in a chain multiply availability down; independent replicas multiply the failure chance down.

- **Series** (a request needs A and B): multiply the availabilities. Every extra dependency on the critical path lowers the total.
- **Parallel** (either replica can serve): multiply the *failure* chances. Two 99% replicas fail together only 0.01% of the time.
- The parallel maths holds only if failures are **independent**. Two replicas in the same rack, on the same power or with the same bad config fail together.

**WORKED EXAMPLE**

A checkout calls the API (99.95%), the payment service (99.9%) and the database (99.95%) in series: 0.9995 × 0.999 × 0.9995 ≈ **99.8%**, about 17.5 hours down a year. Run the database as a primary with a standby in another zone, each 99.95%: the pair is $1 - 0.0005^2 \approx 99.99998\%$, and the chain improves to ≈ **99.85%**. Now the payment service is the weakest link.

## 4 How to improve availability

- **Redundancy** across zones or regions, with failures as independent as you can make them.
- **Remove single points of failure** on the request path.
- **Detect fast, fail over automatically**: health checks and automatic promotion cut MTTR from an hour to seconds or minutes.
- **Timeouts and careful retries**, so a slow dependency doesn't hang everything.
- **Shorten the critical path**: make optional calls asynchronous or degrade gracefully when they fail.

**COMMON MISTAKES**

- Adding dependencies without counting the multiplied availability.
- Assuming replicas fail independently when they share a zone or config.
- Focusing only on MTBF and ignoring recovery time.
- Measuring uptime from a health check while real requests fail.

## 5 Key terms

**Availability**: share of time serving successfully

**MTTR**: mean time to recovery

**Parallel**: any part enough: multiply failures

**MTBF**: mean time between failures

**Series**: all parts needed: multiply availability

**Critical path**: calls a request can't succeed without

## 6 Interview questions

**Q1. Two services at 99.9% each, in series. What's the total?** BASIC
About 99.8%, because you multiply: 0.999 × 0.999 ≈ 0.998, roughly 17.5 hours of downtime a year.

**Q2. What do MTBF and MTTR mean, and how do they give availability?** BASIC
MTBF is how long between failures; MTTR is how long to recover. Availability = MTBF / (MTBF + MTTR).

**Q3. Two independent replicas at 99% each. What's the availability?** BASIC
1 − (0.01 × 0.01) = 99.99%, because the system is down only when both fail at once.

**Q4. You need 99.99% but each server gives 99.5%. What do you do?** SCENARIO
Run several replicas across zones behind a load balancer with fast health checks: two independent 99.5% replicas give $1 - 0.005^2 \approx 99.9975\%$. Then check the rest of the chain (load balancer, database) also meets the target.

**Q5. A service fails once a month and takes an hour to fix. Fastest way to improve it?** SCENARIO
Cut MTTR: automatic failover and good alerts can bring recovery to ~6 minutes, which takes availability from ~99.86% to ~99.986% without changing how often it fails.

**Q6. Your page calls 8 services in series. How do you raise its availability?** SCENARIO
Remove non-essential calls from the critical path (make them async or optional with fallbacks), add redundancy to the weakest services, and use timeouts so a slow dependency degrades the page instead of failing it.

**Q7. Why might two replicas give much less than the calculated 99.99%?** TRICKY
Failures aren't independent: same zone, same power, same bad deploy or config, or a shared dependency like DNS. Correlated failures break the parallel maths.

**Q8. Your dashboard shows 100% uptime but users report errors. How?** TRICKY
The health check measures “process is up”, not “requests succeed”. Measure availability as the success rate of real requests (an SLI), not a ping.

Next: SD-A-011 · Reliability vs Availability vs Durability backendroadmap.com

# Reliability vs Availability vs Durability

*up is not the same as right*

Watch the 3-min video

**In one line:** availability is being up, reliability is being correct over time, and durability is never losing acknowledged data; a system can be fully up and still wrong.

## 1 Three different questions

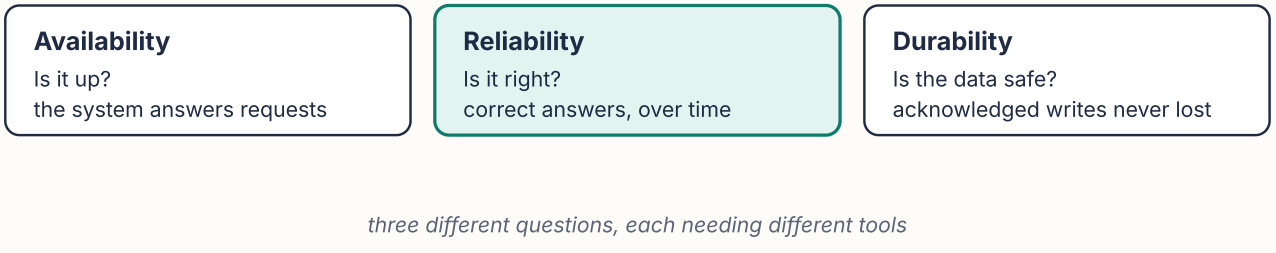



**Figure 1.** A reliable system must also be available, but an available system is not automatically reliable.

## 2 Up, but unreliable

Your banking app opens instantly and every request returns 200 OK, but the balance is wrong. The system is *available*, not *reliable*. Common ways this happens:

- Serving stale data from a broken or never-invalidated cache.
- Wrong prices after a bad deploy or config change.
- Silently dropping writes (a full queue, an ignored error).
- Returning 200 OK with an error message inside the body.




**Figure 2.** Uptime monitoring can be all green while users get wrong answers.

## 3 Measuring reliability

Not just “is it up?” but “is it right?”:

- **Error rates** and failed requests, including errors hidden in 200 responses.
- **Time between failures** (MTBF) for correctness incidents, not only outages.
- **Correctness checks**: reconcile totals (orders vs payments), validate responses end to end, run synthetic transactions.

## 4 Durability: does the data survive?

Durability belongs to storage: once a write is acknowledged, it must never be lost. It's not about reaching the data right now; it's about the data still existing. You get it with replication across disks and zones, checksums to detect corruption, and backups you actually restore from in tests.

**AMAZON S3: DIFFERENT TARGETS FOR DIFFERENT PROPERTIES**

S3 Standard is designed for **99.999999999% durability** (eleven nines): store 10 million objects and you might lose one about once every 10,000 years. But it's designed for **99.99% availability**: occasionally you can't reach a file, yet it's almost certainly still there.

## 5 Tools for each property

| Property | Main tools |
|---|---|
| **Availability** | Redundancy, health checks, fast failover, removing single points of failure |
| **Reliability** | Testing, input validation, idempotent operations, safe deploys (canaries), monitoring correctness, not just uptime |
| **Durability** | Replication across zones, write-ahead logs, checksums, backups, tested restores |

**COMMON MISTAKES**

- Treating 99.99% uptime as “reliable”.
- Backups that were never restored in a test.
- Acknowledging a write before it's safely stored.
- Monitoring only process health, not response correctness.

## 6 Key terms

**Availability**: the system answers

**Durability**: acknowledged data survives

**Checksum**: detects corrupted data

**Reliability**: the answers are correct, over time

**Idempotent**: safe to retry

**Restore test**: proving backups actually work

## 7 Interview questions

**Q1. Can a system be highly available but unreliable?** BASIC

Yes. It can be up and fast while returning wrong or stale data, dropping writes or hiding errors inside 200 responses.

**Q2. What is durability?** BASIC

The guarantee that acknowledged data is never lost, achieved with replication, checksums and tested backups.

**Q3. How is durability different from availability?** BASIC

Availability: can I reach the data now? Durability: does the data still exist? S3 has 11 nines of durability but 4 nines of availability.

**Q4. How would you detect that an e-commerce site is up but wrong?** SCENARIO

Reconcile orders against payments and inventory, run synthetic purchases end to end, alert on business metrics (e.g. orders per minute dropping), and track errors inside responses.

**Q5. Design storage so an acknowledged order is never lost.** SCENARIO
Write to a database that replicates synchronously to another zone (or a quorum) before acknowledging, keep a write-ahead log, take regular backups and restore-test them.

**Q6. A retry charged a customer twice. Which property failed, and how do you fix it?** SCENARIO
Reliability (correctness). Make the payment API idempotent with an idempotency key, so a retried request returns the first result instead of charging again.

**Q7. Do backups give durability?** TRICKY
Only if they're recent, stored elsewhere, uncorrupted and restorable. Untested backups often fail when needed; replication alone doesn't protect against accidental deletes or bad writes.

**Q8. Is replication enough for durability?** TRICKY
It protects against losing a disk or a zone, but it also copies mistakes instantly: a bad delete or corruption replicates too. You still need point-in-time backups.

Next: SD-A-012 · Scalability backendroadmap.com

# Scalability

*load parameters, fan-out and Amdahl's Law*

Watch the 3-min video

**In one line:** scalability means handling more load by adding resources with proportional gains; it starts with picking the right load parameter, and it's limited by serial work and coordination.

## 1 What scalability really means

A system scales if adding resources gives roughly proportional capacity: double the servers, roughly double the load handled. If doubling servers gives 10% more, it doesn't really scale, however many servers you can afford.

## 2 First, define the load

“Load” depends on the system, and choosing the parameter that actually drives cost is the first step of any scaling discussion:

| Load parameter | Example |
|---|---|
| **Requests per second** | An API at 20,000 RPS peak |
| **Read/write ratio** | A catalogue at 100 reads per write |
| **Data size** | 5 TB growing 1 TB a year |
| **Fan-out** | One post delivered to N followers |
| **Concurrent connections** | 1 M open WebSockets for chat |

## 3 The Twitter example

Around 2012 Twitter handled about 4.6 thousand new tweets per second but about 300 thousand timeline reads per second. Building each timeline at read time was too expensive, so Twitter **pre-built timelines on write**: each tweet is copied into every follower's timeline.




**Figure 1.** Fan-out on write: expensive writes, cheap reads. The real load parameter is followers per tweet.

The average user has about 75 followers, but a celebrity has tens of millions, and one tweet would trigger millions of writes. So celebrity tweets are **fetched at read time** and merged in. The lesson: find the

parameter that actually hurts (here fan-out, not tweets per second) and design around it.

## 4 Why scaling is rarely linear

Real systems scale **sublinearly**: the curve bends and flattens because of coordination between machines, shared locks and contention, network overhead, and work that can't run in parallel. Amdahl's Law puts a number on the last one:

```
speedup(N) = 1 / ( s + (1 − s) / N )   ·   s = serial fraction
```

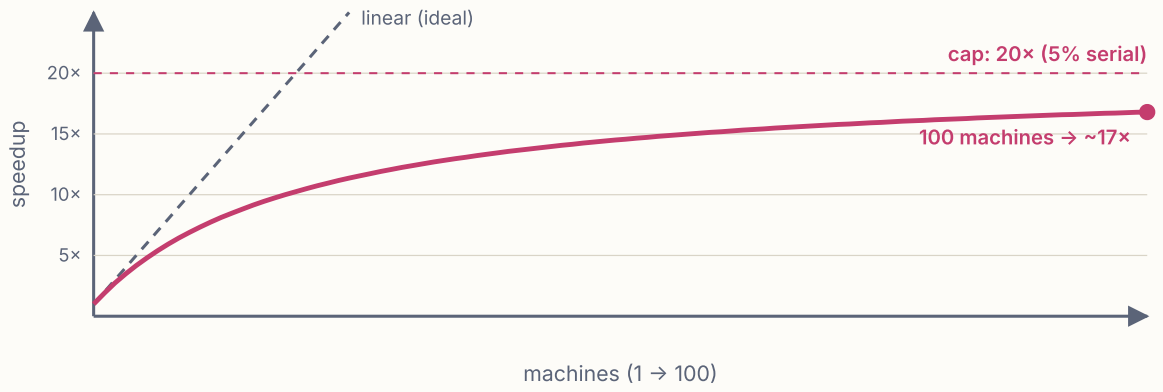



**Figure 2.** With 5% serial work, speedup can never exceed 20×; 100 machines give only about 17×.

**WHAT TO DO ABOUT IT**

Shrink the serial part: avoid global locks, partition data so machines don't coordinate, make work independent per user or per key, and move coordination off the hot path.

**COMMON MISTAKES**

- Saying “scalable” without naming the load parameter.
- Designing for RPS when fan-out or data size is the real driver.
- Expecting linear speedup from more machines.
- One global counter or lock that every request touches.

## 5 Key terms

**Scalability**: more resources → proportionally more capacity

**Load parameter**: the number that drives cost

**Fan-out**: work one action triggers

**Sublinear**: gains shrink as you add machines

**Amdahl's Law**: serial work caps speedup

**Contention**: work waiting on a shared resource

## 6 Interview questions

**Q1. How do you define load for a system?** BASIC

Pick the parameters that drive cost: requests per second, read/write ratio, data size, fan-out, concurrent connections. The right one depends on the system.

**Q2. What limits linear scaling?** BASIC

Serial work, coordination, contention and network overhead. Amdahl's Law: with a serial fraction s, speedup can't exceed 1/s.

**Q3.** **If 90% of the work is parallel, what's the maximum speedup?** BASIC
1 / 0.1 = 10×, no matter how many machines you add.

**Q4.** **Design a news feed for users who follow up to 5,000 accounts. Push or pull?** SCENARIO
Push (fan-out on write) for normal accounts so reads are cheap; pull at read time for accounts with huge follower counts; merge both when building the feed.

**Q5.** **You doubled the servers and throughput rose only 15%. What do you check?** SCENARIO
The shared bottleneck: the database, a global lock, a single-threaded component, a saturated network link, or a downstream rate limit. Profile and find what everyone waits on.

**Q6.** **What load parameter matters most for a chat app?** SCENARIO
Concurrent connections and message fan-out to group members, more than raw requests per second.

**Q7.** **Why did Twitter treat celebrities differently?** TRICKY
Fan-out on write for 30 million followers means 30 million writes per tweet, creating huge delays. Fetching celebrity tweets at read time avoids that spike.

**Q8.** **Is a system that handles 1 M RPS on 1,000 servers scalable?** TRICKY
Not necessarily. Scalability is about the slope: whether adding servers keeps adding capacity proportionally, and at what cost per request.

Next: SD-A-013 · The CAP Theorem backendroadmap.com

# The CAP Theorem

*what it really says, and what it doesn't*

Watch the 3-min video

**In one line:** when the network splits, a distributed system must choose between answering with possibly stale data (availability) and refusing to answer (consistency). Partitions are not optional, so "CA" is not a real choice.

## 1 Precise definitions

"Consistency, availability, partition tolerance: pick two" is catchy, and misleading. The theorem (Brewer's conjecture, proved by Gilbert and Lynch in 2002) uses each word in a narrow sense:

| Property | Meaning in CAP | Not to be confused with |
|---|---|---|
| **C · Consistency** | **Linearizability**: every read returns the latest completed write, as if there were one copy of the data. | The C in ACID (database constraints) |
| **A · Availability** | Every request to a non-failed node gets a non-error response, without a time limit or a "try again". | "99.99% uptime" (an SLO) |
| **P · Partition tolerance** | The system keeps operating when the network drops or delays messages between nodes. | Surviving a crashed server |

## 2 What happens during a partition

Take two replicas, A and B. The link between them breaks. Client 1 writes `x = 2` to A, which cannot tell B. Client 2 then reads `x` from B, which still has the old value 1.

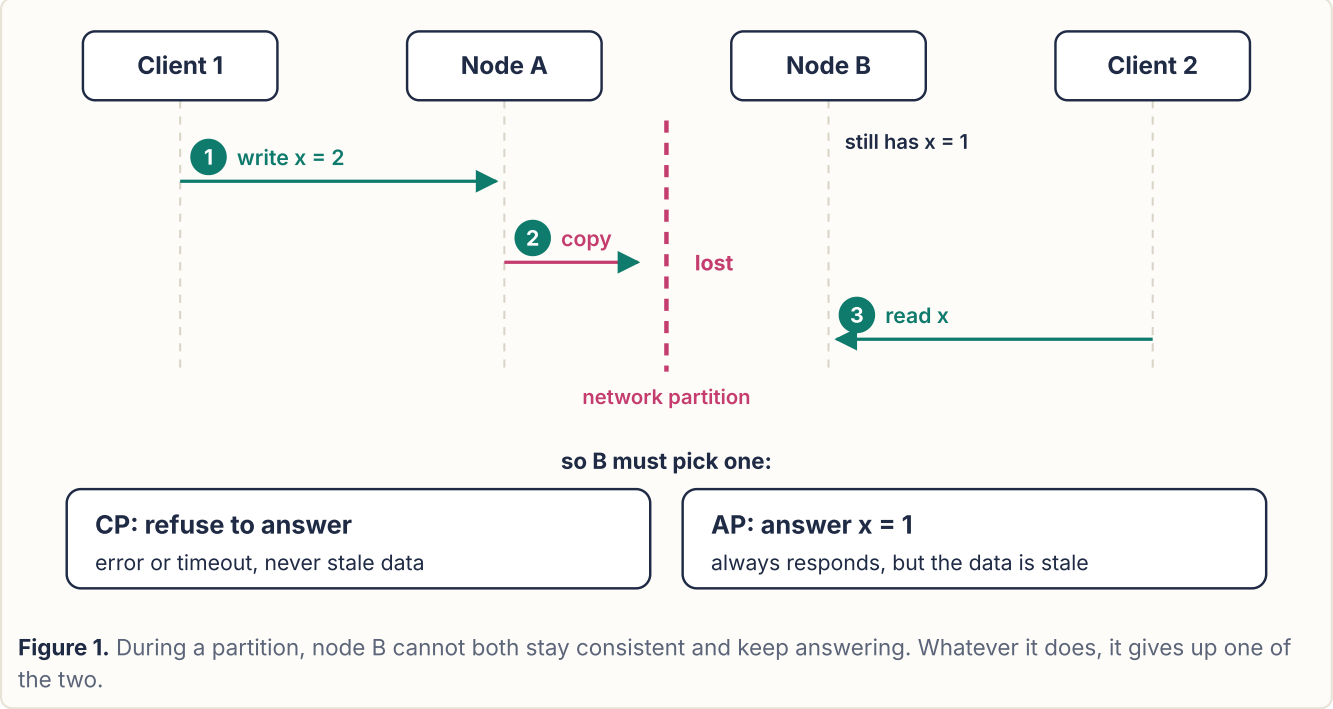



**Figure 1.** During a partition, node B cannot both stay consistent and keep answering. Whatever it does, it gives up one of the two.

That is the whole theorem: **when a partition happens, you choose consistency or availability.** You cannot opt out of partitions, because real networks lose packets, switches fail and cloud zones disconnect. So for any system spread over a network, "CA" describes either a single machine or a system that pretends the network never breaks.

## 3 What CAP does not say

- **It says nothing about normal operation.** With no partition, a system can be both consistent and available. CAP only describes the failure window.
- **It is not a permanent label.** The choice can differ per operation, per table or even per query.
- **It ignores latency.** Even without failures, stronger consistency costs round trips between replicas. That everyday trade-off is what PACELC adds.

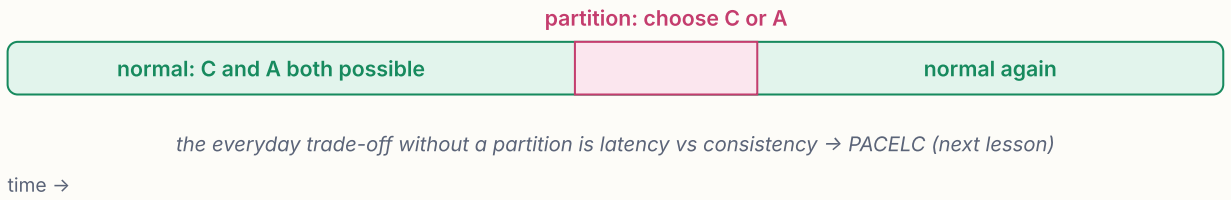



**Figure 2.** Partitions are short, rare windows. CAP describes behaviour inside them; PACELC adds the latency vs consistency choice for the rest of the time.

## 4 CP and AP systems in practice

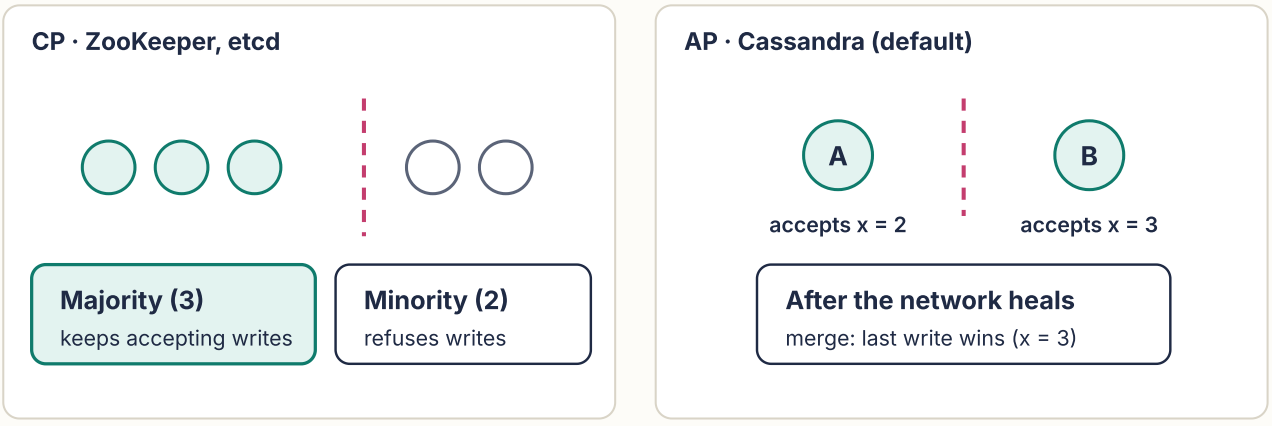



**Figure 3.** Left: a 5-node CP cluster split 3 / 2; only the majority side accepts writes. Right: an AP store accepts writes on both sides and merges them when the network heals (here, by keeping the later write).

| | CP: choose consistency | AP: choose availability |
|---|---|---|
| **During a partition** | The minority side rejects or times out requests | Every reachable node keeps reading and writing |
| **How** | Majority quorum (Raft, ZAB): a write commits only on more than half the nodes | Local writes, then repair: hinted handoff, read repair, last-write-wins or CRDTs |
| **Examples** | ZooKeeper, etcd, Consul; Spanner (CP with very high availability) | Cassandra and DynamoDB in their default modes, DNS, CDNs |
| **Good for** | Leader election, locks, configuration, bank balances, inventory counts | Shopping carts, social feeds, likes, metrics, sessions |
| **Cost** | Errors or waiting during failures | Stale reads and write conflicts to resolve |

**TUNABLE CONSISTENCY (CASSANDRA)**

With 3 replicas (N = 3), each query picks how many replicas must answer: `ONE`, `QUORUM` (2 of 3) or `ALL`. If the replicas written (W) plus the replicas read (R) is greater than N, every read overlaps the latest write: `W + R > N`, for example QUORUM writes and QUORUM reads (2 + 2 > 3). `ONE` / `ONE` is faster and more available, but can return stale data. So one database can behave CP-like for one query and AP-like for the next.

**HOW TO ANSWER IN AN INTERVIEW**

Don't stop at a letter. Describe behaviour: “For the shopping cart I'd stay available during a partition and merge carts afterwards, because a lost sale costs more than a duplicate item. For payments I'd reject writes on the minority side, because a double charge is worse than a short error.”

**COMMON MISTAKES**

- “We'll pick CA”: impossible across a network.
- Treating CAP consistency as ACID consistency.
- Labelling a whole database CP or AP when its behaviour is tunable per query.
- Forgetting that CAP applies only during a partition.

## 5 Key terms

**Linearizability**: reads see the latest write, as if one copy

**Quorum**: a majority of replicas (more than half)

**AP**: answer rather than refuse

**Partition**: nodes up but unable to talk

**CP**: refuse rather than be wrong

**W + R > N**: read and write sets overlap → consistent reads

## 6 Interview questions

**Q1. Explain the CAP theorem precisely.** BASIC

During a network partition, a distributed system cannot provide both linearizable consistency and total availability: a node cut off from the latest write must either refuse the request or return possibly stale data.

**Q2. Why is CA not a real option?** BASIC

Partitions are not optional in a distributed system: networks fail. CA only describes a single node, or a system that assumes the network never breaks. The real choice is how to behave when a partition happens.

**Q3. How is consistency in CAP different from the C in ACID?** BASIC

CAP's C is linearizability: every read sees the latest write across replicas. ACID's C means a transaction keeps the database's rules (constraints, invariants) intact. A single-node database can be ACID-consistent with no replicas at all.

**Q4. Which would you choose for a shopping cart, and for a payment ledger?** SCENARIO

Cart: AP. Keep accepting adds on both sides and merge afterwards; a duplicate item is cheap, a lost sale is not. Ledger: CP. Only the majority side commits; a short error is better than a double spend.

**Q5. You're designing a distributed lock service. CP or AP?** SCENARIO

CP. Two clients holding the same lock is a correctness bug. Use a majority quorum (etcd, ZooKeeper) with leases, and give each lock a fencing token so a client whose lease expired can't keep writing. The minority side refuses lock requests during a partition.

**Q6. A 5-node etcd cluster splits into 3 and 2. What happens on each side?** SCENARIO

The side with 3 has a majority: it keeps or elects a leader and keeps accepting writes. The side with 2 can't reach a quorum, so it rejects writes and linearizable reads. When the network heals, the 2 nodes catch up from the leader's log.

**Q7. Is Cassandra AP or CP?** TRICKY

AP by default, but tunable per query: with QUORUM reads and writes (W + R > N) it returns the latest acknowledged write, and those queries fail when a quorum can't be reached. Describe the behaviour, not just the letters.

**Q8. With N = 3, W = 2 and R = 2, is every read guaranteed to be the latest?** TRICKY

The read and write sets always overlap, so a read reaches at least one replica with the latest acknowledged write. But with last-write-wins timestamps, clock skew can make an older concurrent write win, and a failed partial write can show up later. It's strong in practice, not perfectly linearizable.

Next: SD-A-014 · PACELC Theorem Explained backendroadmap.com

# PACELC Theorem

*CAP's missing half: latency vs consistency*

Watch the 3-min video

**In one line:** PACELC says: during a Partition choose Availability or Consistency; Else, when healthy, choose Latency or Consistency, because keeping replicas in sync always costs waiting time.

## 1 Reading the name

CAP only describes what happens during a network partition, and partitions are rare. PACELC (Daniel Abadi, 2010) covers the other 99%+ of the time. Read it as a sentence: if **P**artition, choose **A** or **C**; **E**lse, choose **L**atency or **C**onsistency.

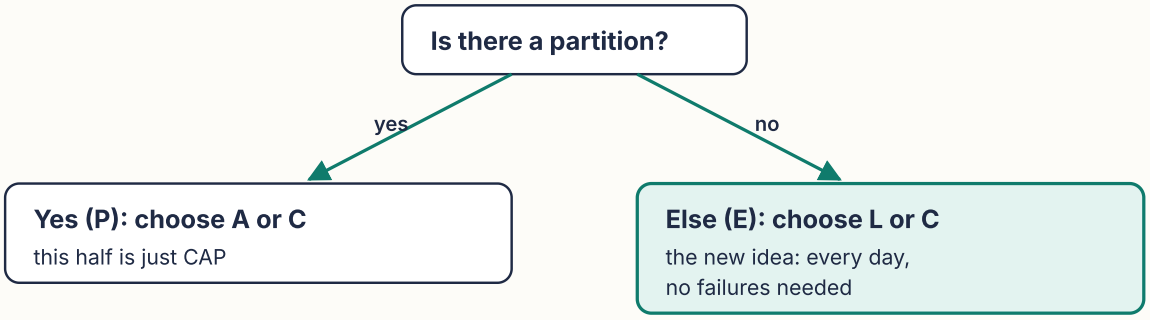



**Figure 1.** PACELC as a decision. The "else" branch applies every day, with no failures at all.

## 2 Why there's a trade-off without failures

Replication. To keep replicas consistent, a write must wait until other replicas confirm it, and waiting costs time. If you don't wait, you answer faster, but a replica may briefly return old data.

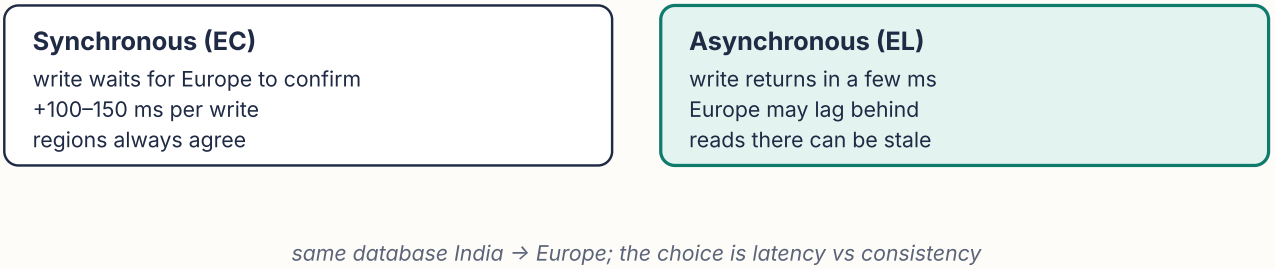



**Figure 2.** Synchronous replication pays latency for agreement; asynchronous pays staleness for speed.

## 3 Classifying real databases

| System | Class | What it means |
|---|---|---|
| **Cassandra** (default) | PA / EL | Available during partitions, fast otherwise; tunable per query |
| **DynamoDB** (default) | PA / EL | Eventually consistent reads by default; strongly consistent reads on request, at extra latency and cost |
| **Google Spanner** | PC / EC | Consistent always; pays with commit latency (TrueTime, cross-region consensus) |
| **PostgreSQL + sync replica** | PC / EC | Writes wait for the standby; no stale reads from it |
| **PostgreSQL + async replicas** | EL on reads | Fast writes; replica reads can lag |

**A SHARPER INTERVIEW ANSWER**

Don't stop at "it's AP". Say what the system does during a partition *and* what it trades in normal operation: "We accept slightly stale reads for low latency on product pages, but reads on the checkout path go to the primary."

**COMMON MISTAKES**

- Thinking consistency only costs something during failures.
- Forgetting cross-region replication adds ~100 ms per synchronous write.
- Labelling a tunable database with one fixed class.
- Using strongly consistent reads everywhere "to be safe" and paying 2× cost.

## 4 Key terms

**PACELC**: Partition: A or C; Else: L or C

**Synchronous replication**: write waits for replicas

**Asynchronous replication**: write returns, replicas catch up

**Replication lag**: how far a replica is behind

**PA/EL**: available + fast, may be stale

**PC/EC**: consistent always, slower

## 5 Interview questions

**Q1. What does PACELC add to CAP?** BASIC

The else case: even with no partition, you trade latency for consistency, because replicas must coordinate to agree.

**Q2. Classify DynamoDB under PACELC.** BASIC

PA/EL by default (eventually consistent reads), with optional strongly consistent reads that move it toward consistency at extra latency and cost.

**Q3. Why is Spanner PC/EC?** BASIC

It keeps strong consistency always, using cross-region consensus and TrueTime, and pays for it with higher commit latency.

**Q4. Your app replicates India → Europe. Users in Europe see old data after edits. Options?** SCENARIO

Make the replication synchronous for that data (EC, +100 ms writes), route the author's reads to the primary region for a while (read-your-writes), or accept the lag where it's harmless.

**Q5. Which reads in an e-commerce site would you make strongly consistent?** SCENARIO

Inventory at checkout, payment status and order confirmation. Product pages, reviews and recommendations can be eventually consistent for speed and cost.

**Q6. When would you pay for synchronous cross-region replication?** SCENARIO

When losing or disagreeing on a write is unacceptable (financial ledgers, critical config) and the business can accept ~100 ms more write latency.

**Q7. Is a single-node PostgreSQL PA, PC, EL or EC?** TRICKY

PACELC is about replicated systems. A single node is consistent but has no replicas to trade against; once you add async replicas, reads from them become EL.

**Q8. Can one system be EL for one query and EC for another?** TRICKY

Yes. Cassandra (consistency level per query) and DynamoDB (consistent-read flag) let you choose per request; describe the behaviour per feature.

Next: SD-A-015 · Consistency Models backendroadmap.com

# Consistency Models

*linearizable vs causal vs eventual, and session guarantees*

Watch the 3-min video

**In one line:** consistency is a spectrum, from linearizable (one copy, costly) to eventual (cheap, may be stale); session guarantees like read-your-writes protect each user in between.

## 1 The spectrum

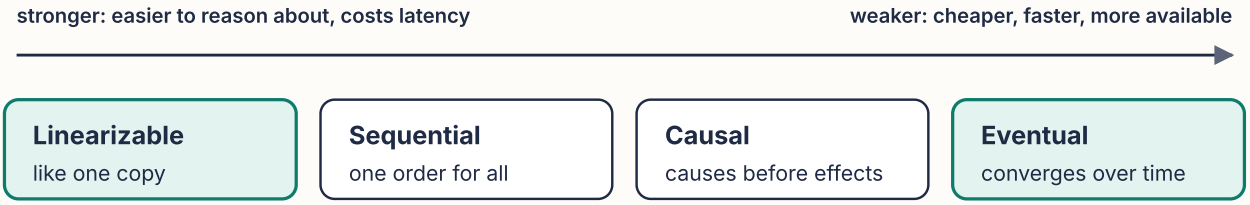



**Figure 1.** Stronger models are easier to reason about but need coordination; weaker ones are cheaper and stay available.

| Model | Promise | Typical use |
|---|---|---|
| **Linearizable** | Once a write completes, every later read (by anyone) sees it, as if there were one copy | Locks, leader election, balances |
| **Sequential** | Everyone sees operations in the same order, but possibly behind real time | Replicated logs |
| **Causal** | If A caused B (a reply to a post), everyone sees A before B; unrelated events may appear in different orders | Comments, chat threads |
| **Eventual** | If writes stop, replicas converge; until then reads may be stale | Likes, view counts, feeds |

## 2 Session guarantees

Between the extremes, session guarantees protect a single user's experience:

- **Read-your-writes:** you always see your own changes.
- **Monotonic reads:** once you've seen a newer value, you never see an older one again.
- **Monotonic writes:** your writes are applied in the order you made them.
- **Writes-follow-reads:** a reply you write is ordered after the post you read.

## 3 Why read-your-writes matters

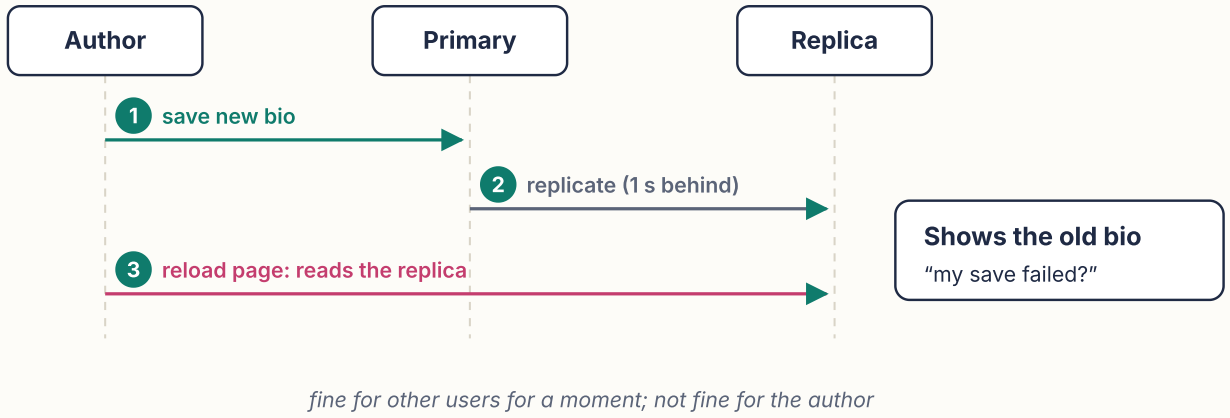



**Figure 2.** The write goes to the primary, the reload reads a lagging replica, and the author sees the old value.

**THREE WAYS TO PROVIDE IT ON REPLICAS**

1. **Read your own data from the primary** for a short window (e.g. 10 s) after you write.
2. **Track a version:** remember the log position (LSN) of the user's last write in the session, and read only from a replica that has caught up to it.
3. **Pin the user to one replica** to get monotonic reads, so time never goes backwards for them.

**COMMON MISTAKES**

- Treating consistency as a yes/no choice.
- Using eventual consistency for data the author just edited.
- Load balancing reads across replicas so a user sees values jump back and forth.
- Paying for linearizability where causal or eventual is enough.

## 4 Key terms

**Linearizable**: behaves like one copy

**Eventual**: replicas converge over time

**Monotonic reads**: never see an older value again

**Causal**: causes are seen before effects

**Read-your-writes**: you see your own changes

**Replica lag**: how far behind a replica is

## 5 Interview questions

**Q1. Linearizability vs eventual consistency?** BASIC

Linearizable: every read sees the latest completed write, as if there were one copy. Eventual: replicas converge only over time, so reads may be stale.

**Q2. What is causal consistency?** BASIC

Operations that are causally related (a reply and its post) are seen in the same order by everyone; unrelated operations may be seen in different orders.

**Q3. How do you provide read-your-writes on replicas?** BASIC

Route the author's reads to the primary for a short window, or read only from replicas that have caught up to the version of their last write.

**Q4. Comments sometimes appear before the post they reply to. Which model fixes it?** SCENARIO

Causal consistency: track the dependency (the reply references the post's version) and don't show the reply until the post is visible on that replica.

**Q5. A user's balance flickers between two values on refresh. Why, and how do you fix it?** SCENARIO

Reads hit different replicas with different lag. Provide monotonic reads by pinning the user to one replica, or read balances from the primary.

**Q6. Which consistency model for a like counter, and for a seat reservation?** SCENARIO

Likes: eventual (a few seconds stale is harmless). Seat reservation: linearizable (two people must never get the same seat).

**Q7. Does eventual consistency mean data can be lost?** TRICKY

No. It's about when replicas agree, not durability. An acknowledged write is still stored; some replicas just see it later.

**Q8. Is linearizable the same as serializable?** TRICKY

No. Linearizability is about single-object reads and writes being up to date in real time. Serializability is about transactions over many objects behaving as if run one at a time. Strict serializability is both.

Next: SD-A-016 · Fault Tolerance backendroadmap.com

# Fault Tolerance

*fault vs failure, zones and blast radius*

Watch the 3-min video

**In one line:** faults (a disk dies, a bad deploy) are certain; fault tolerance stops them becoming failures (users affected) through redundancy, isolation and fast detection.

## 1 Fault vs failure

A **fault** is one component going wrong: a disk fails, a process crashes, a dependency slows down. A **failure** is the whole system no longer serving users. Fault tolerance means stopping faults from turning into failures.

| Kind of fault | Examples |
|---|---|
| **Hardware** | Dead disks, bad memory, power or network loss |
| **Software** | Bugs, memory leaks, a dependency that slows to a crawl, a retry storm |
| **Human** | A wrong config, a bad deploy, a deleted table. In practice a large share of outages. |

## 2 How to tolerate faults

- **Redundancy and replication:** more than one copy of every important thing.
- **Isolation:** a problem in one part can't spread to the rest.
- **Fast detection:** health checks, heartbeats and timeouts move traffic away from broken parts automatically.

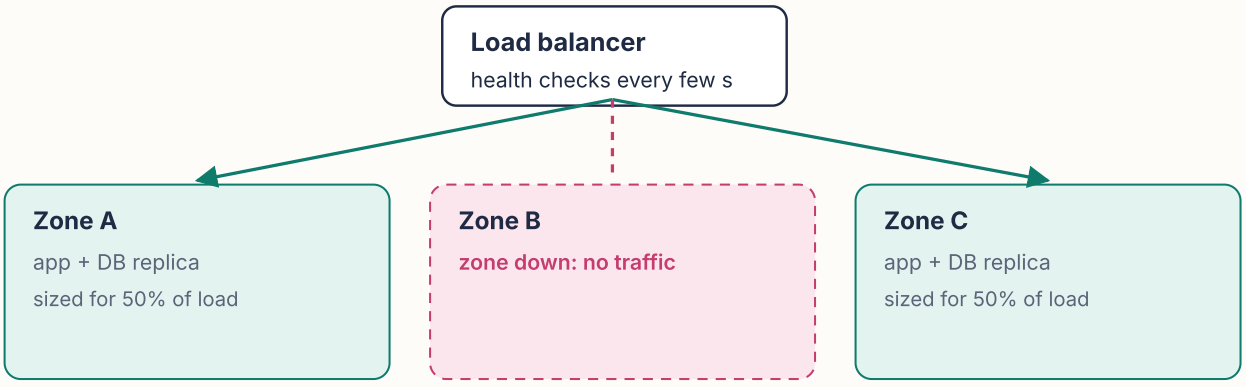



**Figure 1.** Replicas in three availability zones. If one zone goes down, the load balancer stops sending it traffic and the other two carry the load.

**PLAN CAPACITY FOR A ZONE LOSS**

With three zones at peak, each normally carries a third of the traffic. If one fails, the other two must carry half each. So each zone must be sized for ~50% of peak, about 1.5× its normal share. Without that headroom, losing one zone overloads the other two and turns a fault into a failure.

## 3 Blast radius

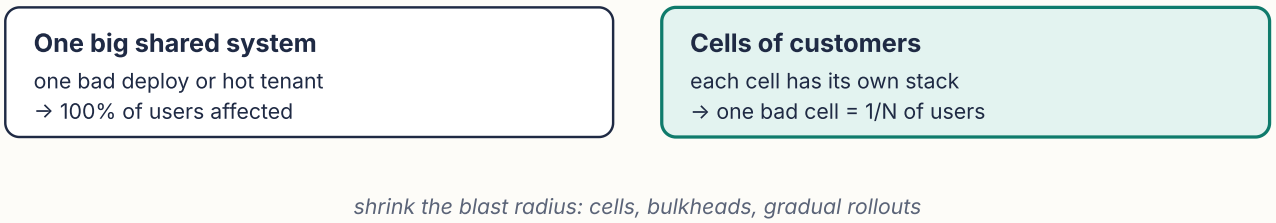



**Figure 2.** Cells limit how many users one problem can reach.

- **Cells or shards of customers:** each slice of users gets its own stack; a bad cell affects only its slice.
- **Bulkheads:** separate thread or connection pools per dependency, so one slow dependency can't use up all the workers.
- **Gradual rollouts:** canary to 1% of traffic, watch the metrics, then widen, with automatic rollback.

**THE MINDSET: DESIGN FOR FAILURE**

Assume every component will fail. For each box ask “what happens when this dies?”, then test it on purpose with chaos experiments (kill an instance, cut a zone, slow a dependency) before reality does.

**COMMON MISTAKES**

- Three zones, each sized only for a third of peak.
- All replicas sharing one config or deploy that breaks them together.
- No timeouts, so one slow dependency hangs every request.
- Big-bang deploys to 100% of users.

## 4 Key terms

**Fault**: one component misbehaves

**Failure**: users are affected

**Availability zone**: separate data centre in a region

**Blast radius**: how much breaks when something fails

**Bulkhead**: isolated pool per dependency

**Chaos engineering**: breaking things on purpose to learn

## 5 Interview questions

**Q1. What's the difference between a fault and a failure?** BASIC

A fault is one component misbehaving; a failure is the system no longer serving users. Fault tolerance stops faults from becoming failures.

**Q2. How do you limit blast radius?** BASIC

Isolate with cells and bulkheads, spread across zones, and roll out changes gradually with canaries and automatic rollback.

**Q3. What are the three kinds of faults?** BASIC

Hardware (disks, power, network), software (bugs, leaks, slow dependencies) and human (bad configs and deploys).

**Q4. You run in 3 zones. How much capacity does each need?** SCENARIO
Enough to survive losing one: each zone sized for about half of peak, so the remaining two can carry everything.

**Q5. A slow payment provider makes your whole API time out. Fix?** SCENARIO
Timeouts on the call, a bulkhead (separate pool) for the payment client, a circuit breaker that fails fast, and a fallback such as queueing the payment for retry.

**Q6. A config push took down every server at once. How do you prevent it?** SCENARIO
Treat config like code: validate it, roll it out gradually (one cell or 1% first), watch error rates, and roll back automatically.

**Q7. Do more replicas always mean more fault tolerance?** TRICKY
Only against independent faults. Replicas that share a bug, config, zone or dependency fail together; diversity and isolation matter as much as count.

**Q8. Why run chaos experiments in production?** TRICKY
Staging rarely matches production's traffic, data and dependencies. Small, controlled experiments in production (with a kill switch) reveal real weaknesses before an outage does.

Next: SD-A-017 · Single Point of Failure backendroadmap.com

# Single Point of Failure (SPOF)

*find the weak link, then remove it*

Watch the 3-min video

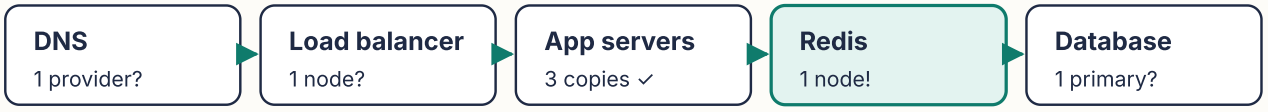
**In one line:** a single point of failure is any component whose loss takes the whole system down; find them by walking the request path, remove them with active-passive or active-active redundancy.

## 1 What a SPOF is

A chain of ten strong links breaks at its one weak link. A **SPOF** is any component that exists once and that everything depends on. Redundancy everywhere else doesn't help if one piece has no backup.

They hide in obvious places (a single database, load balancer or region) and in surprising ones: a single DNS provider, one TLS certificate about to expire, one deploy pipeline, one engineer who holds the keys.

## 2 Finding them




*walk the path and ask of every box: if this dies, does the system go down?*

**Figure 1.** The app tier is redundant, but sessions live on one Redis node. When it dies, every user is logged out at once.

**FIXING THE REDIS SPOF**

Add a replica in another zone with **Redis Sentinel** to detect failure and promote it automatically (usually within seconds), or run **Redis Cluster** with replicas per shard. Clients must reconnect to the new primary, so use a Sentinel-aware client.

## 3 Two redundancy patterns

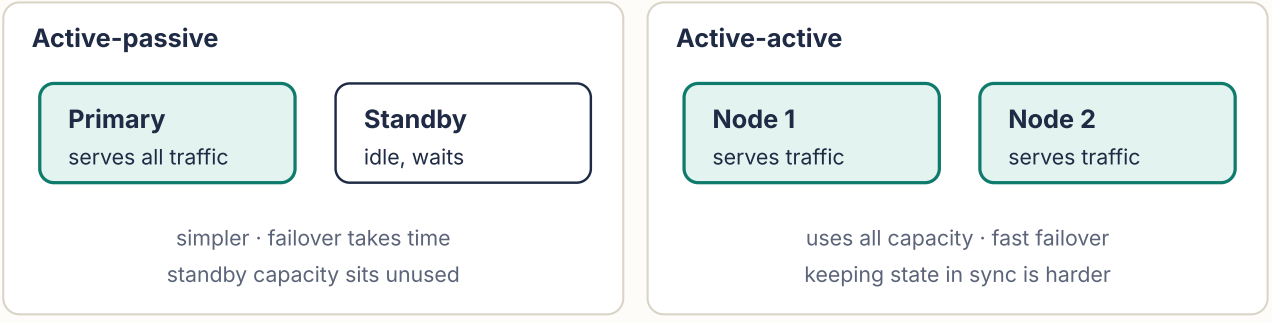



**Figure 2.** Active-passive is simpler; active-active uses capacity better but needs state kept in sync across nodes.

| | Active-passive | Active-active |
|---|---|---|
| **Traffic** | One node at a time | All nodes at once |
| **Failover** | Seconds to minutes (detect + promote) | Near-instant: others keep serving |
| **Cost** | Standby idle | All capacity used |
| **Hard part** | Promotion without split brain | Concurrent writes and conflicts |
| **Typical use** | Primary database + standby | Stateless app servers, multi-region reads |

## 4 The database tier

- A primary with replicas in other zones, and **automatic failover** that promotes a replica when the primary dies.
- **Backups** stored elsewhere, with tested restores.
- At larger scale, multi-region, or a distributed database designed to survive losing nodes.

**COMMON MISTAKES**

- Redundant app servers in front of one database or cache.
- One load balancer, or one DNS provider.
- Forgetting non-server SPOFs: certificates, secrets, CI/CD, a single person.
- A standby that was never actually failed over to in a test.

## 5 Key terms

**SPOF**: one component whose loss stops everything

**Active-active**: all nodes serve at once

**Redis Sentinel**: monitors Redis and promotes a replica

**Active-passive**: standby takes over on failure

**Failover**: switching to a backup

**Split brain**: two nodes both think they're primary

## 6 Interview questions

**Q1. How do you find SPOFs?** BASIC

Walk the request path (DNS, load balancer, app, cache, database) and ask of every component: if this dies, does the system go down? Include certificates, secrets, pipelines and people.

**Q2. How do you remove a SPOF in the database tier?** BASIC

Add replicas in other zones with automatic failover, plus tested backups; at scale, consider multi-region or a distributed database.

**Q3. Active-passive vs active-active?** BASIC

Active-passive: one serves, a standby takes over (simpler, slower failover, idle capacity). Active-active: all serve (better use, faster failover, harder state sync).

**Q4. Every user was logged out when one Redis node restarted. Fix it.** SCENARIO

Sessions had one home. Add a replica with Sentinel for automatic failover, or Redis Cluster; or use signed session tokens so the cache isn't needed to stay logged in.

**Q5. Is the load balancer itself a SPOF?** SCENARIO

It can be. Use a managed cloud load balancer (redundant by design) or a pair with a floating IP (VRRP/keepalived) and health checks, and spread across zones.

**Q6. Only one engineer knows how to deploy. Why is that a SPOF, and what do you do?** SCENARIO

If they're unavailable, nobody can ship or roll back. Automate the pipeline, document runbooks, and rotate on-call so several people can deploy.

**Q7. What's split brain, and how do you prevent it?** TRICKY

After a partition, two nodes both act as primary and accept conflicting writes. Prevent it with a majority quorum for promotion and fencing (the old primary is blocked from writing).

**Q8. Can a region be a SPOF?** TRICKY

Yes. A region-wide outage takes everything there down. Multi-region removes it, at the cost of replication latency, consistency trade-offs and higher spend; use it when the requirement justifies it.

Next: SD-A-018 · Simple Beats Clever backendroadmap.com

# Simple Beats Clever

*cost, performance and complexity trade-offs*

Watch the 3-min video

**In one line:** every component costs money, operational effort and new failure modes; design for your real scale plus reasonable growth, compare designs by total cost of ownership, and keep the simplest one that meets the numbers.

## 1 Every box has a price

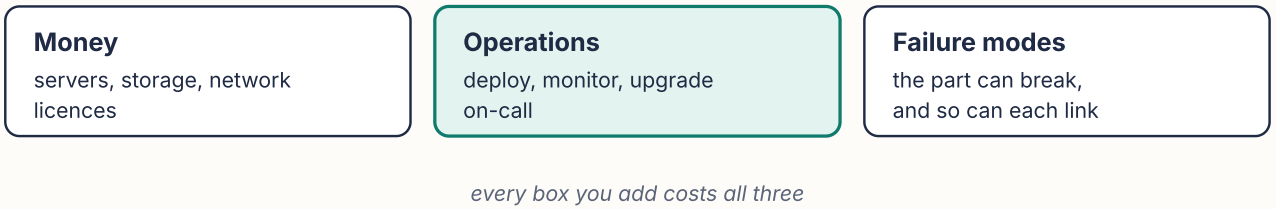



**Figure 1.** The cloud bill is the smallest part of what a component costs.

The real cost is the **total cost of ownership (TCO)**: infrastructure plus engineering time, on-call pain and slower feature work. A design that saves 10% on servers but needs two more engineers to run it is not cheaper.

## 2 A worked example

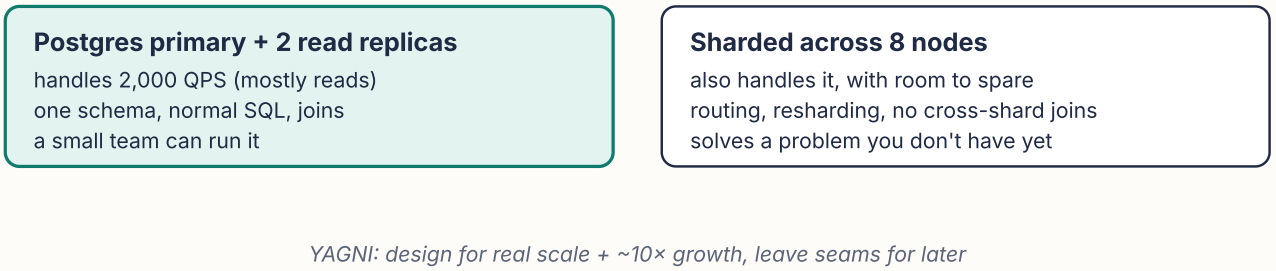



**Figure 2.** Both designs meet the load; the simple one does it with far less to build and run.

## 3 YAGNI: you aren't gonna need it

Design for your real scale plus reasonable growth, perhaps 10×, not 100×. Leave clear **seams** (a data-access layer, an interface in front of the store) so you can add caching or sharding later, when the numbers demand it.

**TRIGGERS TO ADD COMPLEXITY**

Add sharding when the primary's write load or data size nears its limits even after scaling up; add a cache when read latency or database load misses targets; split a service when a team or a scaling need demands it. Each trigger is a number, not a feeling.

## 4 Estimating cost per request

**WORKED EXAMPLE**

**Load:** 2,000 queries/s × ~2.6 M seconds a month ≈ **5.2 billion requests a month**.
**Monthly cost:** add up servers, storage, network and the share of engineering time.
**Divide:** say $6,000 / 5,200 million ≈ **$1.15 per million requests**. Now two designs can be compared with real numbers instead of opinions.

## 5 Performance has a price curve

Going from 500 ms to 100 ms may be cheap: an index, a cache, fewer round trips. Going from 10 ms to 2 ms can cost a fortune: more memory, more regions, specialised hardware. Spend where users actually feel the difference, and stop when the requirement is met.

**COMMON MISTAKES**

- Big-tech architecture on day one: microservices, Kafka, sharding for 1,000 users.
- Comparing designs on cloud cost only.
- Optimising latency far beyond what users notice.
- No seams, so the simple design can't grow later.

## 6 Key terms

**TCO**: total cost: infrastructure + people + risk

**Seam**: an interface that lets you swap parts later

**Operational load**: work to deploy, monitor, fix

**YAGNI**: don't build what you don't need yet

**Cost per request**: monthly cost ÷ monthly requests

**Over-engineering**: complexity beyond the requirements

## 7 Interview questions

**Q1. When is a simpler design better, even if it scales less?** BASIC
When it meets the real load with headroom: it's cheaper to run, easier to operate, and has fewer failure modes.

**Q2. How do you estimate the cost of a design?** BASIC
Estimate the load, add up monthly infrastructure and people costs, and compute cost per request or per million requests.

**Q3. What is YAGNI in system design?** BASIC
Don't build for scale or features you don't need yet; design for real scale plus ~10× growth, and leave seams to extend later.

**Q4. 2,000 QPS, mostly reads. Shard or not?** SCENARIO
Not yet: a Postgres primary with read replicas (and maybe a cache) handles it comfortably. Shard when write load or data size approaches a single primary's limits.

**Q5. Your team proposes Kafka for 50 events a second. Your view?** SCENARIO
Probably overkill: a database table or a managed queue handles 50/s with far less to operate. Kafka pays off at high throughput, replay or many consumers.

**Q6. Product wants p99 from 40 ms to 10 ms. How do you decide?** SCENARIO

Ask what users or revenue gain, estimate the cost (memory, regions, rework), and compare. If 40 ms already feels instant, spend elsewhere.

**Q7. Isn't designing for 100× scale the safe choice?** TRICKY

No. It costs money and complexity now, slows feature work, and the real bottlenecks at 100× are hard to predict. Seams plus measured triggers are safer.

**Q8. Can a simpler design be less reliable?** TRICKY

Sometimes; a single database is a SPOF. Simple doesn't mean no redundancy: keep replicas and failover, just avoid complexity the requirements don't ask for.

Next: SD-A-019 · Stateless vs Stateful backendroadmap.com

# Stateless vs Stateful

*sessions, sticky sessions and scaling WebSockets*

Watch the 3-min video

**In one line:** stateless services keep nothing between requests, so any instance can serve anyone and they scale freely; push state into a few purpose-built stateful systems that replicate and rebalance.

## 1 Two kinds of components




**Figure 1.** Stateless components are interchangeable; stateful ones hold data or connections that live on a specific node.

Stateless services scale easily: add instances, remove instances, lose one in a crash, and nothing is lost. The load balancer can send any request anywhere, and autoscaling just works.

## 2 Making a service stateless

**FASTAPI SESSIONS**

The app keeps sessions in an in-memory dictionary. With two pods, half the requests land on the pod that doesn't know the user. Move sessions to Redis (or use signed tokens like JWTs), and every pod becomes interchangeable.

| Option | How | Trade-off |
|---|---|---|
| **Shared session store** | Sessions in Redis, keyed by a cookie | One more system to run (make it redundant) |
| **Signed tokens** | State inside a signed JWT cookie | No store, but hard to revoke; keep tokens small |
| **Sticky sessions** | Load balancer pins a user to one server | Quick fix; uneven load, sessions lost when that server dies |

## 3 What stateful systems need

- **Placement:** which node owns which data or which user (e.g. consistent hashing).
- **Replication:** copies, so a node can die safely.
- **Rebalancing:** moving data or connections when nodes join or leave.

## 4 Scaling a chat app's WebSockets

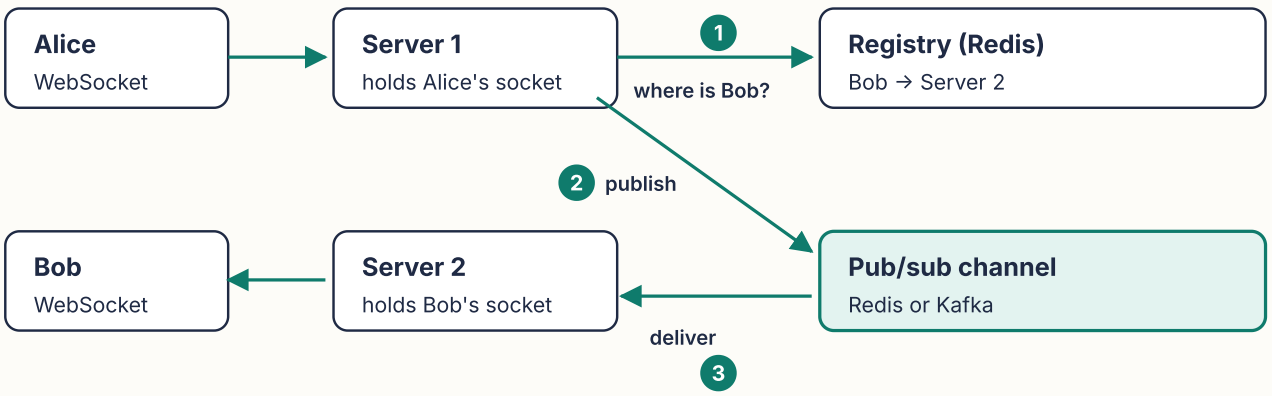



**Figure 2.** Alice and Bob are connected to different servers. A registry finds Bob's server; pub/sub carries the message there.

1. Each user holds a long-lived WebSocket to one server; the server records *user → server* in a registry with a TTL heartbeat.
2. When Alice sends a message, Server 1 looks up Bob and publishes to Server 2's channel (or a per-user/per-room channel).
3. Server 2 pushes it down Bob's socket. Messages are also saved to the database, so a client that reconnects can fetch what it missed.
4. If a server dies, its clients reconnect (with backoff and jitter) to another server, which updates the registry.

**THE RULE OF THUMB**

Keep compute stateless, and push state into a few systems built to replicate and rebalance: databases, caches, message brokers. Scale the easy parts freely and the hard parts carefully.

**COMMON MISTAKES**

- In-memory sessions on horizontally scaled servers.
- Sticky sessions as the long-term answer.
- Broadcasting every chat message to every server.
- Reconnect storms: all clients reconnecting at once with no jitter.

## 5 Key terms

**Stateless**: nothing kept between requests

**Sticky session**: LB pins a user to one server

**Pub/sub**: publish once, subscribers receive

**Stateful**: data or connections live on a node

**JWT**: a signed token that carries state

**Rebalancing**: moving data when nodes change

## 6 Interview questions

**Q1. Why are stateless services easier to scale?** BASIC

Any instance can serve any request, so you can add, remove or lose instances freely and the load balancer can route anywhere.

**Q2. How would you scale a stateful WebSocket server?** BASIC

Add servers behind a load balancer, track which user is on which server, route messages between servers with pub/sub, and let clients reconnect on failure.

**Q3. What do stateful systems need that stateless ones don't?** BASIC

Placement (who owns what), replication (survive node loss) and rebalancing (move data when nodes change).

**Q4. After adding a second server, some chat messages never arrive. Why?** SCENARIO

The sender's server only knows its own sockets; the recipient is on another server. Add a user→server registry and pub/sub between servers.

**Q5. Sessions in Redis or JWTs?** SCENARIO

Redis: easy revocation and small cookies, but a dependency to keep redundant. JWTs: no store, but revocation is hard and tokens grow. Many systems use short-lived JWTs plus a refresh token in a store.

**Q6. A WebSocket server with 100K connections crashes. What happens next?** SCENARIO

Its clients reconnect to other servers; use exponential backoff with jitter to avoid a stampede, update the registry, and fetch missed messages from the database.

**Q7. When are sticky sessions acceptable?** TRICKY

As an optimisation (better cache locality) when losing the session is harmless, or briefly during a migration; not as the only place state lives.

**Q8. Is a cache stateful?** TRICKY

Yes, it holds data, but it's usually treated as disposable: losing it should only cost performance (cache misses), never correctness. Plan for cold-start load on the database.

---